TU Clausthal

Master Thesis

by

Kaan Akgün

Matriculation Number: 533140

Model Driven Development of Operational Design Domain for AI Based Systems

1st Supervisor:	**Prof. Dr.-Ing. Umut Durak**
2nd Supervisor:	**Prof. Dr. S. Hartmann**

25th March 2025

Institute of Informatics
Clausthal University of Technology

Declaration of Authorship

I have read and understood the guidelines of the Technical University of Clausthal, including those regarding the use of literature and other sources. I confirm that I have prepared this thesis independently by myself. Any information taken from other sources and being reproduced in this thesis is clearly referenced.

In terms of the general examination regulations, this work has not yet been submitted to any other examination division.

I hereby agree that my master thesis may be exhibited in the institute's library and kept for inspection.

Clausthal-Zellerfeld, 25th March 2025	
Location, Date	Kaan Akgün

Abstract

The aviation industry is undergoing a major transformation through the adoption of Artificial Intelligence (AI). Current AI-based systems in the aviation domain offer a promising alternative by enabling simulations that ensure safety compliance; however, they also highlight the need for regulations established by authorities such as the European Union Aviation Safety Agency (EASA). The concept of The Operational Design Domain (ODD) is used to ensure safe operations of the AI-based autonomous systems. ODD specifies the precise operating conditions, such as environmental, geographical, and performance parameters, under which an AI-based system is intended to function safely. Model-driven development (MDD) provides a practical approach to simulate operational scenarios and system interactions, allowing users to assess AI-based systems under realistic conditions.

The Informatics department of TU Clausthal developed a tool called Operation Domain Modeling Environment (ODME) tool for modeling complex systems using System Entity Structure (SES) in the MDD approach. The SES framework, a hierarchical modeling approach that organizes system components, their attributes, and relationships in a structured manner, enables efficient pruning and scenario generation. The SES framework in the tool allows modeling pruning of the model to create Pruned Entity Structure (PES), generate specific executable scenario models, and specify the ODD of the modeled system.

Generating numerous scenario models is error-prone, emphasizing the need for automation. ODD is decoupled in the current MDD approach, which complicates the preparation for the automated scenario generation process. Integrating ODD into scenario generation enhances scenario diversity evaluation and ensures parameter coverage. Furthermore, data output should be optimized for the size of the data while maintaining integrity and consistency for the automated scenario generation process.

In this thesis, the Partial Pruning Entity Structure (PPES) is introduced as an intermediate layer between SES and PES to enhance the preparation of an automated scenario generation process in MDD by integrating ODD into the scenario generation process. PPES, a refinement approach, organizes model components by eliminating re-

dundant entities and optimizing the variables of the entities. The option of converting from text to binary format using Protocol Buffers (Protobuf) serves as an alternative to meet the requirements of this process.

The case study within the drone domain model demonstrates that PPES facilitates the preparation of an automated scenario generation process, and Protobuf ensures data integrity while reducing data size.

Overall, the proposed framework integrates realistic operational constraints into a structured modeling process, laying preparation for future automated AI-driven aviation solutions.

Zusammenfassung

Die Luftfahrtindustrie befindet sich durch den Einsatz von Künstlicher Intelligenz (KI) in einem tiefgreifenden Wandel. Die aktuellen KI-basierten Systeme in der Luftfahrt bieten eine vielversprechende Alternative, da sie Simulationen ermöglichen, die die Einhaltung der Sicherheitsvorschriften gewährleisten; sie zeigen jedoch auch die Notwendigkeit von Vorschriften, die von Behörden wie der European Union Aviation Safety Agency (EASA) festgelegt wurden. Das Konzept der Operational Design Domain (ODD) wird verwendet, um den sicheren Betrieb von KI-basierten autonomen Systemen zu gewährleisten. ODD spezifiziert die genauen Betriebsbedingungen, wie Umwelt-, geografische und Leistungsparameter, unter denen ein KI-basiertes System sicher funktionieren soll. Die Model-driven development (MDD) bietet einen praktischen Ansatz zur Simulation von Betriebsszenarien und Systeminteraktionen, der es den Benutzern ermöglicht, KI-basierte Systeme unter realistischen Bedingungen zu bewerten.

Das Fachgebiet Informatik der TU Clausthal entwickelte ein Werkzeug namens Operation Domain Modeling Environment (ODME) zur Modellierung komplexer Systeme unter Verwendung der System Entity Structure (SES) im MDD-Ansatz. Das SES-Framework, ein hierarchischer Modellierungsansatz, der Systemkomponenten, ihre Attribute und Beziehungen in einer strukturierten Weise organisiert, ermöglicht ein effizientes Pruning und die Generierung von Szenarien. Das SES-Framework im Werkzeug ermöglicht das Pruning des Modells, um eine Pruned Entity Structure (PES) zu erstellen, spezifische ausführbare Szenarienmodelle zu generieren und die ODD des modellierten Systems zu spezifizieren.

Die Generierung zahlreiche Szenariomodelle ist fehleranfällig, was die Notwendigkeit einer Automatisierung unterstreicht. ODD sind im derzeitigen MDD-Ansatz entkoppelt, was die Vorbereitung auf die automatisierte Szenariogenerierung erschwert. Die Integration von ODD in die Szenariengenerierung verbessert die Bewertung der Szenarienvielfalt und gewährleistet die Abdeckung der Parameter. Darüber hinaus sollte die Datenausgabe für die Größe der Daten optimiert werden, während gleichzeitig die Integrität und Konsistenz für den automatisierten Szenariengenerierungsprozess erhalten bleibt.

In dieser Masterarbeit wird die Partial Pruning Entity Structure (PPES) als Zwischenschicht zwischen SES und PES eingeführt, um die Vorbereitung eines automatisierten Szenariogenerierungsprozesses in MDD durch die Integration von ODD in den Szenariogenerierungsprozess zu verbessern. PPES, ein Verfeinerungsansatz, organisiert die Modellkomponenten durch die Eliminierung redundanter Entitäten und die Optimierung der variablen Grenzen der Entitäten. Die Möglichkeit des Übergangs vom Text- zum Binary format unter Verwendung von Protocol Buffers (Protobuf) dient als Alternative, um die Anforderungen dieses Prozesses zu erfüllen.

Die Fallstudie innerhalb des Drohnen-Domänenmodells zeigt, dass PPES die Vorbereitung des automatisierten Szenario-Generierungsprozesses erleichtert und Protobuf die Datenintegrität gewährleistet, während die Datengröße um reduziert wird.

Insgesamt integriert das vorgeschlagene Rahmenwerk realistische betriebliche Einschränkungen in einen strukturierten Modellierungsprozess und bereitet so zukünftige automatisierte KI-gesteuerte Luftfahrtlösungen vor.

Acknowledgements

I wish to convey my deepest appreciation to everyone who has encouraged me throughout the course of my Master's program.

First and foremost, my thanks go to my supervisor, Prof. Dr.-Ing. Umut Durak, for his unwavering guidance, motivation, and insightful comments. His expertise and support have played a crucial role in defining my research and academic growth.

I am also profoundly thankful to my advisor, Siddhartha Gupta, for his steadfast support and direction.

My gratitude extends to my parents and my girlfriend for their constant love and patience during the highs and lows of this journey.

Thank you all for being integral to this experience.

Contents

List of Figures

List of Abbreviations

AI - Artificial Intelligence

ASAM - Association for Standardization of Automation and Measuring Systems

BSI - British Standards Institution

CoDAN - Concepts of Design Assurance for Neural Networks

ConOps - Concept of Operations

DEEL - DEpendable & Explainable Learning

EASA - European Union Aviation Safety Agency

ISO - International Organization for Standardization

MDD - Model Driven Development

OD - Operational Domain

ODD - Operational Design Domain

ODME - Operational Domain Modeling Environment

PES - Pruned Entity Structure

PPES - Partial Pruning Entity Structure

Protobuf - Protocol Buffers

SAE - Society of Automotive Engineers

SES - System Entity Structure

XML - eXtensible Markup Language

YAML - YAML Ain't Markup Language

1 Introduction

1.1 Motivation

In recent years, research in applied artificial intelligence (AI) within the aviation industry has achieved remarkable progress [20]. This development has consequently seen significant regulatory developments in recent times; most notably, the European Union Aviation Safety Agency has come forth with a concept paper and developed regulations for machine learning applications to ensure the safe integration of AI technologies [10]. The operational design domain (ODD) is the important aspect marked within this framework and should form one of the pivot bases in order to provide safety operations of AI-based autonomous systems.

The concept of ODD was first developed for the automotive industry, and it has been defined by the SAE J3016 standard as "the set of conditions under which a given driving AI function is designed to operate safely" [26]. These include environmental, geographical, and temporal constraints, including dynamic elements present or absent, such as traffic and road characteristics [28]. Using the ODD for aviation involves the modification of such concepts to specific operational scenarios to make sure AI systems operate within well-defined safety boundaries [5]. The development and strict observance of a sound ODD reduce risks inherent in field operation with AI-based systems [3].

The EASA stresses the integral linking of the Concept of Operations (ConOps) to ODD as a base for comprehensive safety in aviation [32]. ConOps is a document that describes high-level operational scenarios from the user's view, while ODD outlines the system's specific operating conditions. This ensures that a systematic approach towards safety will be made possible because the logical and concrete test cases are basic in simulation-based validation [32]. It allows users to ensure the safety of the system by validating scenario models [16].

The Informatics department of TU Clausthal developed a tool called Operation Domain Modeling Environment (ODME) tool for modeling complex systems using System Entity Structure (SES) in the MDD approach. Model-Driven Development (MDD) uses

abstraction to manage complexity in systems engineering, representing system components at a higher level while allowing these components to be expressed with software models, thus identifying potential problems and optimization opportunities by modeling their interactions; finally, it creates a structural map of the system, making the integration of new requirements [8]. The SES framework, a hierarchical modeling approach that organizes system components, their attributes, and relationships in a structured manner, enables efficient pruning and scenario generation. The SES framework in the tool allows modeling pruning of the model to create Pruned Entity Structure (PES), generate specific executable scenario models, and specify the ODD of the modeled system [27],[23].

In ODME, an XML-based format is used for data output in the modeling and scenario generation process [4]. XML was chosen to ensure that data is portable across platforms and stored in a structured manner. This format allows the representation of hierarchical data structures and provides a standard language for defining and generating simulation scenario models [4]. However, XML may have problems such as not providing integrity in the preparation of automated scenario generation process and high data size.

Manually creating numerous scenario models is prone to errors, underscoring the crucial role of automation. Integrating the ODD into the scenario generation process enhances the evaluation of scenario diversity and ensures comprehensive parameter coverage. However, in the existing MDD approach, the decoupling of ODD complicates the preparation of automated scenario generation process. Additionally, data output of scenario models should be optimized to efficiently manage size while maintaining integrity and consistency throughout the automated scenario generation process.

Research Question

The research questions that guide this thesis are the following:

How can scenario generation process be optimized through a methodological approach that can resolve the decoupled structure of ODD within the MDD process by determining operational parameters, thereby preparing for an automated scenario generation process in AI-based aviation systems?

How can the data output of scenario models be optimized to effectively manage size while ensuring integrity and consistency for the preparation of automated scenario generation process?

The following objectives have been established to tackle this research questions:

- PPES (Partial Pruning Entity Structure) serves as an intermediate layer between SES and PES to ensure effective integration of the ODD in the MDD process. Resolving the decoupled structure of ODD in the existing MDD process facilitates its integration into the preparation of automated scenario generation process. The integration of ODD into scenario generation process enhances scenario diversity evaluation and ensures parameter coverage , thereby ensuring that AI and autonomous systems are thoroughly assessed across diverse and realistic scenarios..
- PPES facilitates a refinement approach by organizing model components and eliminating redundant parameters through a partial pruning technique. In this method, parameters are maintained within certain boundaries rather than being fully specified, allowing PPES to generate model variants within these boundaries. This sets the stage for the automated scenario generation process through its integration with ODD.
- PPES facilitates the tracking of different model variants with time-stamped version control. Each PPES variant is tagged with a unique timestamp. Furthermore, scenario data is in binary format alternative with Protobuf instead of text-based formats (XML/YAML). This optimization ensures data integrity and consistency by reducing file size.

1.2 Thesis Outline

- **Chapter one** is an introduction that sets the groundwork for the study's main focus and objectives. It defines the main issue the research aims to address and introduces the main research questions guiding the research.
- **Chapter two** delves into the study's background, providing a review of relevant theoretical concepts. It explores the intersection of AI and MDD in the aviation context and current studies. The background discussion examines core elements

such as domain models like SES and ODD, scenario models like PES, and behavior modeling. The chapter underscores the importance of these elements in forming a coherent framework for modeling AI-based systems and sets the stage for understanding the detailed processes discussed in later chapters.

- **Chapter three** explains the methodology and presents a case study. It details how PPES is created and integrated to better align operational parameters with scenario generation requirements. It wraps up with a practical case study on an aviation simulation model, showing how PPES optimizes the scenario generation process in ODME.
- **Chapter four** discusses the implications of the findings and limitations of the study.
- **Chapter five** wraps up the study by highlighting the conclusion, main findings, quantitative results, comparative analysis with existing methods, and suggestions for future work. It discusses how successfully integrating ODD improved model optimization and data handling. The chapter recommends further automating the scenario generation process of PPES.

2 Background

This chapter examines the critical role of scenario generation in aviation and AI-based systems, aiming to provide an in-depth understanding of current methodologies and standards in these areas. The concept of scenarios and the scenario-generation process will be emphasized. Current research on AI-based systems in aviation and the industry standards for these systems will be reviewed. Additionally, the relationship between AI-based systems and MDD will be evaluated. The main components of the MDD process, including the ontology and metamodel concepts used in operational scenarios, as well as the domain model and its components SES, OD, ODD, and the scope of ODD will be discussed. Moreover, PES and Behavior Trees will be explored in scenario modeling. Finally, the roles of ODME and Protobuf will be reviewed as an alternative data output.

A scenario is a description of the beginning and progress of events [9].

Scenario Abstraction

The scenario development process includes three levels of abstraction: operational, conceptual, and executable scenarios. Operational scenarios define the initial conditions and expected outcomes [9]. Conceptual scenarios specify the world in detail in the system simulation. Executable scenarios include all the information required for the preparation and execution of simulation environments [29].

Operational scenarios form the basis of the scenario generetion process. These scenarios, usually provided by users in the early stages of the simulation development process, include the starting and ending states and the transition paths from these states [9].

Detailed definitions, such as conceptual scenarios, are needed as the development process progresses. This transformation, made by simulation experts, describes the representative world in detail in the simulation environment [29].

In the final stage, conceptual scenarios are transformed into executable scenarios. These scenarios provide all the information needed for the preparation, startup, and execution of the simulation environment and can be directly processed by member applic-

ations [29]. This phase is usually performed by application operators in the simulation environment, with the support of simulation experts [9]

2.1 Ongoing Studies Regarding AI-Based Systems in Aviation

AI will likely bring revolutionary changes to aviation, just as in many other fields. In any case, safe and reliable integration of these technologies into the aviation industry is rapidly being advanced through various research efforts and collaborations.

EASA is leading industry action with its Artificial Intelligence Roadmap, which itself contains the goal to enable autonomous commercial air transport operations over controlled airspace and a limited number of conditions by 2035. This roadmap identifies AI trustworthiness analysis, learning assurance, explainability, and safety risk mitigation as foundational building blocks that require deeper review [1].

Collaboration with Daedalean has led to publishing a set of reports referred to as "Concepts of Design Assurance for Neural Networks" or CoDAN [6]. The first report covers the W-shaped learning assurance lifecycle of neural networks and generalization. The second report elaborates even more about those and on the concept of explainability [7].

EASA has also classified AI/ML applications into three levels: Level 1 for human assistance, Level 2 for collaboration human/machine, and Level 3 for autonomous machines. Each level has different safety and certification requirements; guidelines have been published accordingly [10].

EUROCAE WG-114 was established in 2019. In collaboration with SAE, this working group is developing the AS6983 standard, which will support the development and certification of aeronautical systems using the newly created Machine Learning Development Lifecycle (MLDL) [19].

In 2021, WG-114 and SAE G-34 published their report "Artificial Intelligence in Aero-

nautical Systems: Statement of Concerns" on the categorization of AI techniques, performing gap analysis for existing standards, and making recommendations for future actions by the industry [14].

The DEEL research program was put together to build knowledge on explainability and robustness issues surrounding AI algorithms. It shall deliver a profound assessment of challenges that have to be resolved if any subsequent certification processes are to be performed [33].

2.2 Model-Driven Development

Model-driven approaches focus on representing systems and processes, utilizing formal scenario and ODD modeling structures. These approaches employ ontologies and metamodels to define system components and their relationships, organizing system dynamics more effectively. SES and Behavior Trees are used to model the dynamic content of scenarios and ODDs. MDD places models at the heart of the development process by creating abstract representations and enabling their transformation into code through automated or semi-automated processes. MDD features a high level of abstraction, automation, and continuous improvement. Standardization and separation of concerns are also key components of these methods, aiming to manage complexity and enhance software quality [8].

2.3 Relation between AI-based Systems and MDD

Sprockhoff et al. highlighted [27] that AI-based systems and MDD are complementary fields in state-of-the-art systems engineering. The contribution of AI-based approaches is significant in functional and non-functional requirements, while MDD supports design and verification at the architectural level by structured models. AI addresses functional and non-functional requirements and architectural design, while MDD contributes to managing these with systematic frameworks, automatic test case generation, model checking, and safety analysis.

Model-driven approaches are valuable tools for the development of AI-based systems. MDD helps in modeling, tracking, and verification of the functional requirements of any system. Due to the complexity of AI-based systems, the semi-formal nature of MDD

supports the detection of software errors and design flaws early. Model-driven approaches offer methods of systematically deriving test cases from models for AI algorithms [27].

Verification of Non-Functional Requirements

AI systems should be verified regarding functional requirements and non-functional ones, such as performance, reliability, and safety. MDD simplifies the identification and evaluation of hazardous scenarios. Redundancy and monitoring patterns are also implemented to improve system safety [27].

Optimized Architecture Comparison and Selection

Architecture directly reflects performance and reliability in AI-based systems. MDD allows for the analysis of various architectural options, including trade-off analyses and techniques of variant management. This will be crucial in choosing a system with the most suitable AI components. Comparisons based on MDD will offer better design decisions by considering the complexity of AI systems [27].

Framework utilization

Model-driven frameworks are well used in most MDD-integrated AI cases. Systemic design frameworks define and guide proper model creation, updates, analysis, development, and utilization. These supporting frameworks allow reviewing and defining entire AI-based systems [27].

2.4 Operational Scenarios

2.4.1 Meta Level

The metamodel, using taxonomy and ontology at the meta-level, provides an in-depth framework that allows defining the elements of the operational domain models in a structured and organized way. This baseline structure will let all information about the operational environment be captured and modeled coherently and correctly. This

metamodel is an accurate guide with clear relation and classification for generating valid and valid operational scenarios, effective simulation, and testing processes [22].

Ontology

Ontology formally represents knowledge, and it fulfills concepts and their relationships within a domain modal [22].

Ontologies can model knowledge about scenes. Applying ontologies for that enables automation in the scenarios of testing and simulation of airborne systems, as it automates these scenarios and is highly efficient compared to manual scenario creation by experts [22].

Moreover, an ontology metamodel provides a structured and standard mechanism to organize and represent environmental knowledge and vehicle capabilities. Thus, ODD and ontology metamodels could be considered a prerequisite to assure the safety and performance of the systems under study [2].

2.5 Domain Model

Following the metamodel, the domain model is the pragmatic extensible version of the structured framework provided at the meta-level. In domain modeling, identifying all operational scenarios involves recognizing all elements within those scenarios, including their attributes and relationships. A complete model represents all elements of the scenarios, with these elements being represented by entities. The relationships among these entities include aspects, multi-aspects, and specialization [10].

2.5.1 SES

SES is a high-level, complex ontology for representing knowledge about the decomposition, taxonomy, and coupling of complex systems introduce a formal framework for modeling these systems in a hierarchical and structured manner [21].

SES is defined as a high-level ontology for representing knowledge related to system coupling, decomposition, and taxonomy and is viewed as an advancement in system theory-based modeling and simulation approaches [21]. Utilizing the interactions

between decomposition, coupling, and taxonomies [24] facilitates a precise specification of a model family [25]. Additionally, SES functions as a formal ontology framework that delineates the components of a system and their hierarchical relationships [35].

SES Elements

Zeigler et al. [35] cites SES is made up of four basic elements that define its structure and the relationships inside the system:

Entity: The system's tangible or intangible component is associated with specific variables and states [4].

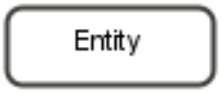

Figure 2.1: Entity

Aspect: Aspect designates the decomposition relationship of an entity node [4].

Figure 2.2: Aspect

Specialization: Specialization represents entity taxonomy that indicates categories or families of different variations an object can assume [4].

Figure 2.3: Specialization

Multi-Aspect: The multi-aspect element is used to specify a particular type of Aspect that describes a multiplicity relationship, indicating that a parent entity is composed of several entities of the same type [4].

Figure 2.4: Multi-aspect

SES Axioms

SES is governed by several axioms uniformity, strict hierarchy, alternating mode, valid sibling relationships, attached variables, and inheritance. Its modeling and simulation theory basis, coupled with clarity and minimal axioms [34].

SES maintains its consistency and coherence through the use of basic axioms. Nodes with the same label and their subtrees will be isomorphic, ensuring uniformity and strict hierarchy prevent any label from occurring more than once along a path. The alternating mode requires that if a node is an entity representation, its successor has to be classified either as an aspect or as a specialization. To preserve valid sibling relationships, no two sibling nodes can have the same label. Furthermore, the attached variables indicate that variable types for the same item must have distinct names. Inheritance, as it allows a specialization to inherit variables and features from its parent entity [34].

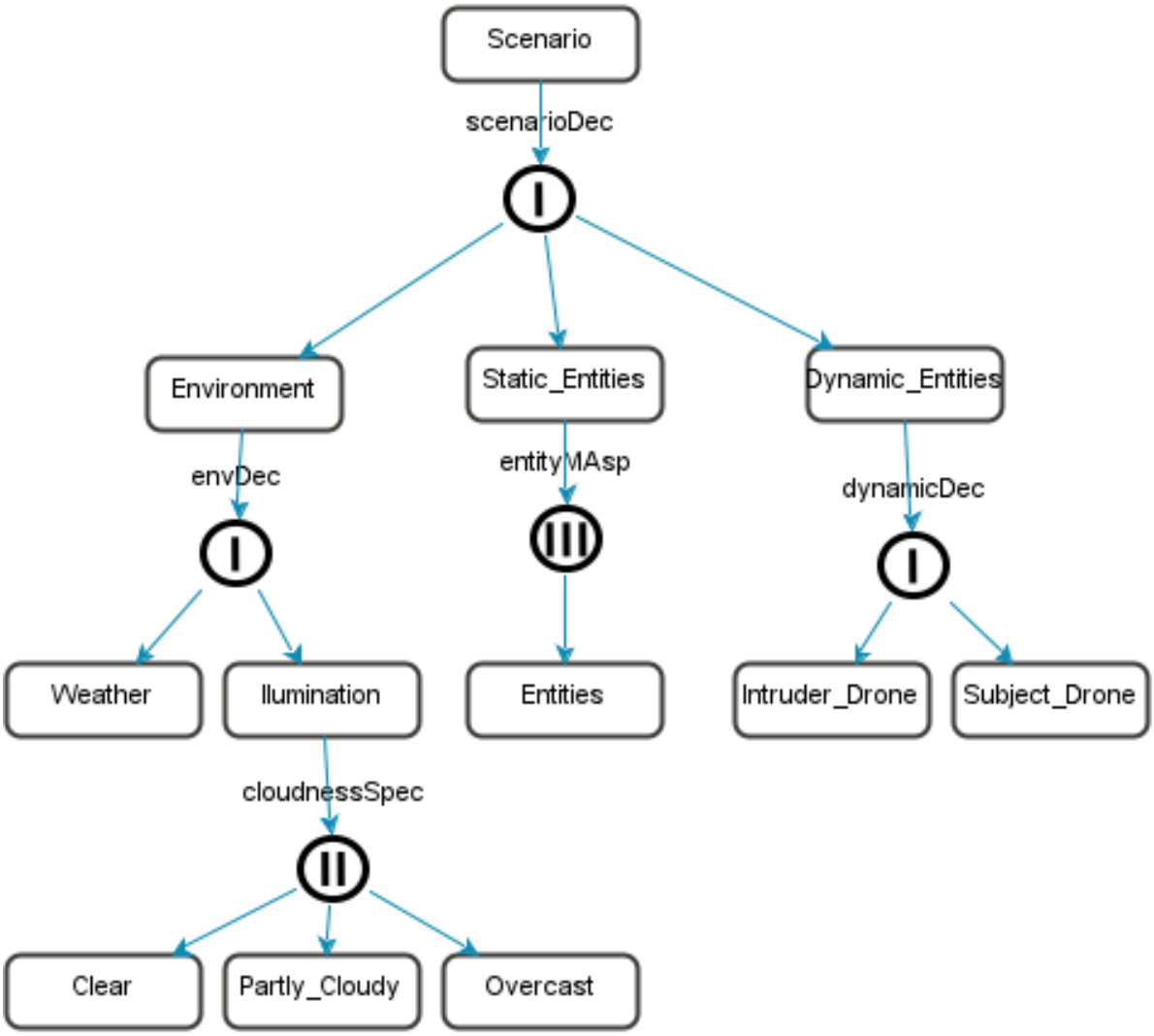

Figure 2.5: SES Model

Figure 2.5 represents an SES model that identifies environmental conditions, static entities, and dynamic entities.

Computational representation using XML enables the integration and structuring of SES models [34].

2.5.2 Operational Domain (OD)

Conditions in which a specific AI-based system is designed to operate effectively, following the established ConOps. These conditions may include environmental, geographical,

and temporal restrictions [10].

2.5.3 ODD

The ODD specifies the operating parameters, including their range and distribution, within which the AI component is intended to function. The system will operate effectively only when these parameters are met. Additionally, the ODD takes into account the interdependencies among operating parameters, allowing for the refinement of ranges as needed; this means that the range of one or more parameters may be influenced by the value or range of another parameter [10].

Application concept of ODD in the aviation domain can help in characterizing and bounds the states of the operational environment relevant for aviation, reflecting conditions, location, and/or dynamic factors of potential safety: [10], [18]. These may further be specified to include scene items, environmental, and parameters of airborne systems [18]. Incorporation of these into SES-based modeling frameworks ensures regular reflection of ODD attributes with regard to operating limits in field conditions [10].

OD and ODD

While the OD is a general operational environment, the ODD formulates specific constraints under which AI systems can operate in a safe environment. That is, ODD is a subset of OD, determined through processes in model-driven development (MDD). For example, OD could refer to general restrictions in airspace, while ODD addresses specific values that can be operated in a safe environment through an AI system [10].

ODD Attributes

ODD attributes are essential components that define the operational environment for flight systems [10].

- **Scene (Scenery):** These are physical aspects of the environment where the vehicle operates, such as road types, buildings, etc.

- **Environmental Conditions:** These are the external physical conditions that must be considered for vehicle operation, including weather, visibility, and network connectivity.
- **Geographical Boundaries:** Specific areas where the system can function.
- **Dynamic Elements:** The moving elements in the environment, such as other airborne systems.

ODD in the aviation context is aimed at identifying the safe operational range for airborne systems. It defines specific attributes related to aviation, such as airspace class and aircraft state parameters [10].

Safety Importance of ODD

ODD is crucial for assuring safety in aviation operations. The operating design domain helps identify the conditions under which the aircraft operates securely, thereby reducing the risk of accidents [17].

Enhancing safety assurance in AI-based systems hinges on three key improvements: continuous monitoring against ODD constraints, derivation of operational scenarios from Concepts of Operations ConOps, and alignment with international standards. Continuous monitoring ensures compliance within safety boundaries, enabling early issue detection and proactive interventions during critical tasks. By deriving operational scenarios, the framework supports comprehensive validation that captures diverse conditions and edge cases. Finally, adherence to standards like ASAM OpenODD and EASA guidelines reinforces safety protocols and facilitates regulatory acceptance in the aviation industry [30].

ODD Coverage

Gupta et al. [15] propose a five-stage method for assessing ODD coverage, which includes reading the target ODD, restricting the domain model based on the ODD, and dividing parameters into ”buckets” according to discrete and continuous values. Structural and parameter coverages are averaged to calculate overall coverage, providing a detailed evaluation of how well the scenario generation conforms to ODD constraints.

2.6 Scenario Modelling

2.6.1 PES

PES is an essential process in SES model simplification, mainly by refining or discarding elements of lesser or no importance. The systematic pruning process begins with identifying the redundant elements based on the objectives of the specific analysis. An evaluation of the component interaction in a model exists, thus considering the relationships and dependencies these components have. These are then made more manageable through pruning of elements that are in excess, notwithstanding the requirement for maintaining coherence in a model. Complete testing and validation of the pruned models ensure that the best performance of these models results in satisfying constraints and meeting requirements as previously established [4].

2.6.2 Pruning Procedures

Naja et al. mention that [23] PES have different ways of optimization, such as Multi-Aspect Node Pruning, Specialization Node Pruning, and Entity Variable Pruning.

A description of these three pruning methods will follow.

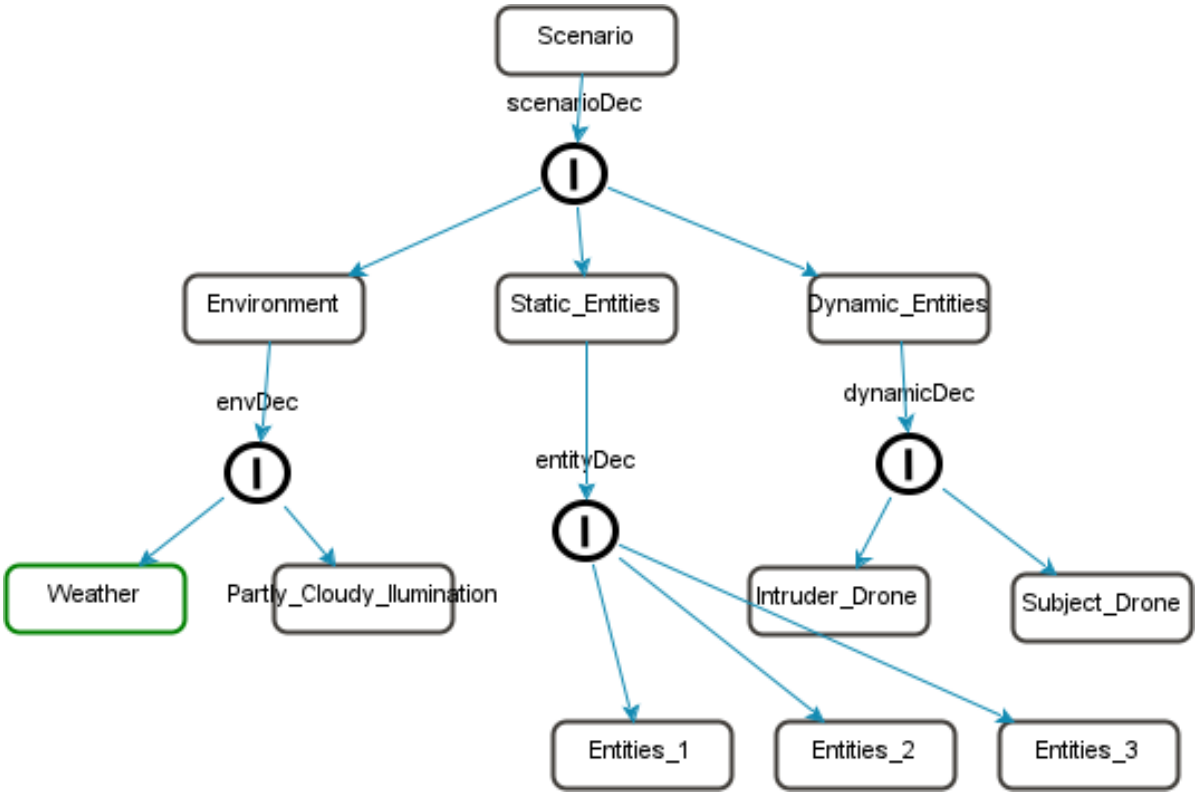

Figure 2.6: PES Procedures

Multi-Aspect Pruning

Multi-aspect node pruning means handling and optimizing different aspects inside the SES model. When a Multi-Aspect node must be pruned, one cardinality or the overall count of its aspects needs to be defined first. In the estimation, it is assumed that a definite number of certain kinds of aspects have been created.

It evaluates multi-aspect nodes in the model, including aspects that are functional, necessary, or contributory to the whole structure. Based on this evaluation, all the unnecessary or redundant multi-aspect nodes get pruned. In this pruning process, a certain number of aspects are generated according to a specified cardinality previously decided upon by the user. Figure 2.6, pruning the ”entity-MAsp” entity with a cardinality value of three will generate three entities.

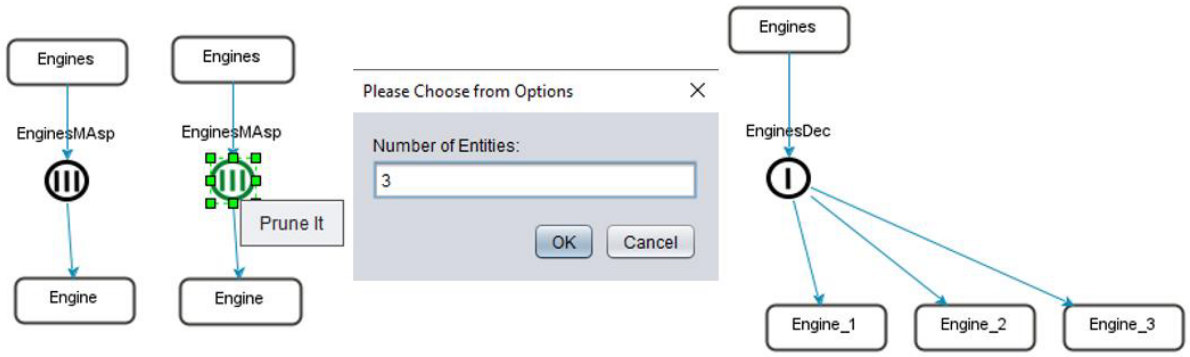

Figure 2.7: Multi-Aspect Pruning

Specialization Pruning

Specialization Node Pruning refers to identifying and removing customizations whose resulting behaviors are deemed ineffective within the process in an SES model representation. Specialization nodes in the model are reviewed. Each customization's relationship to the parent entity and contribution to the model's overall structure are scrutinized.

There needs to be a child node choice to make a variation about the demands on customization, or there needs to be an analysis made after which nodes shall be pruned that do not add functionality nor are considered fundamental to the model. The above-stated nodes get the nodes for Specialization. In figure 2.6, here are three varieties for the element "cloudnessSpec," namely: Clear, Partly Cloudy, and Overcast. This leaves only "Partly Cloudy" after pruning. The name for this resulting entity, after pruning, is "Partly Cloudy Illumination." This assures a specific model.

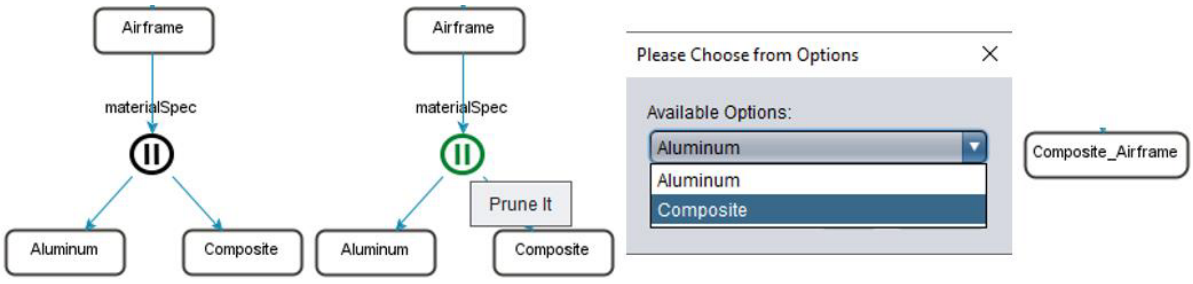

Figure 2.8: Specialization Pruning

Entity Variable Pruning

Entity Variable Pruning The goal of entity variable pruning is to perform the pruning by updating the values of variables associated with entities. Entities with variables are eligible for pruning and will be highlighted in green in the scenario modeling mode. Pruning is done by updating the values of the variables of an entity. Figure 2.6, after pruning the ”precipitation rate” variable of the ”Weather” entity, the updated value of this variable is calculated and set. Once the entity variable has been pruned, the variable table itself will also get updated with that new value to keep the variable values of the model up to date and the same.

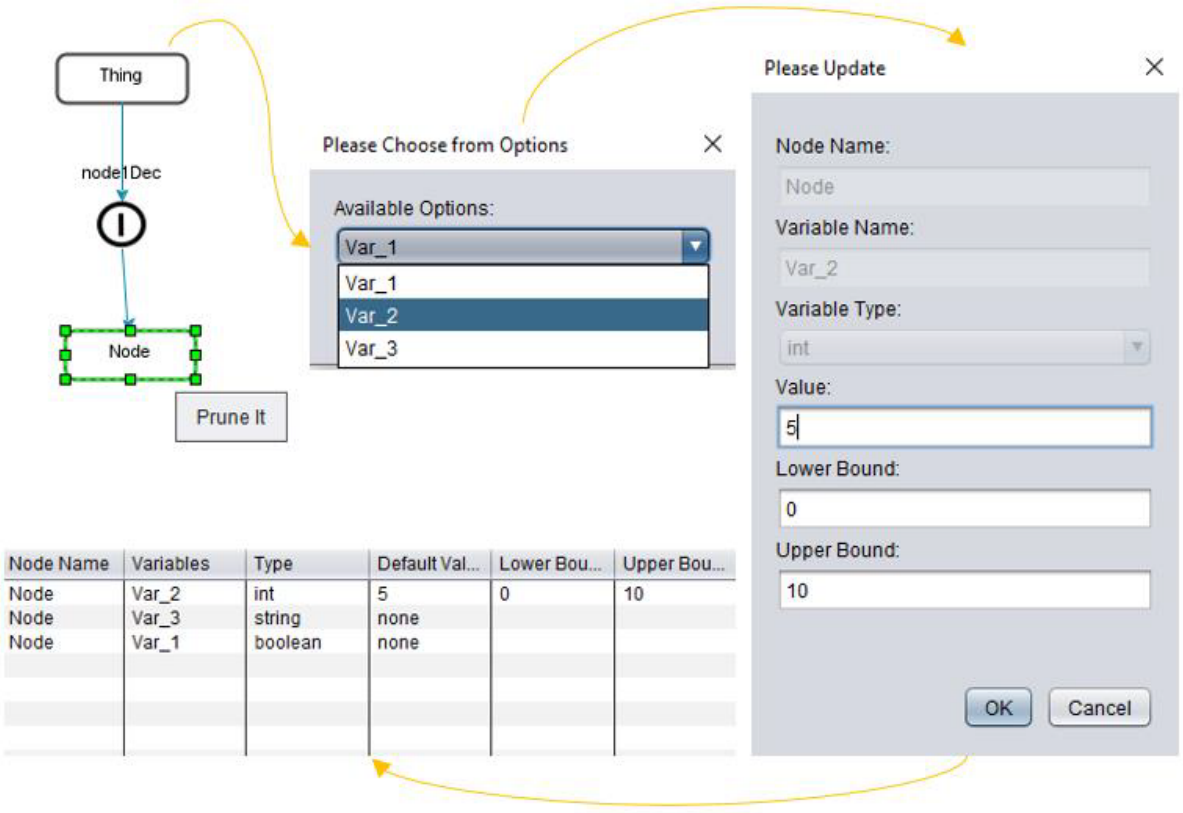

Figure 2.9: Entity Variable Pruning

2.6.3 Behavioral Modeling

Hemel et al mention [31] behavioral modeling can define and represent the dynamic behavior of a system according to various operational scenarios. Behavioral modeling

becomes imperative to gain insight into the system's reaction during operation with regard to varied input and condition factors.

Salient features of behavioral modeling include the following:

Dynamic Representation

Behavioral models characterize how the system behaves in time, how its state changes, and how it interacts with other systems, showing how it will respond? This is necessary to simulate real-life conditions that the system may experience during its operation.

Scenario Development

Behavioral modeling is also linked with establishing operational, conceptual, and executable scenarios that define the initial state, the actions to be taken, and the desired end state. Thus, a framework is allowed to test how the system would behave in those situations.

Verification and Validation

With the application of behavioral models, developers can inspect and verify if the system meets its requirements in performing as it should under various conditions. This is particularly so in safety-critical domains like aviation, where failures may have consequences. Integration with Simulation Tools Behavioral models are often integrated with simulation tools to create a realistic testing environment.

2.7 Operational Design Modeling Environment - ODME

The ODME is a structure developed for complex systems modeling by Karmokar et al. and the Informatics department of TU Clausthal [4], which allows users to create a visual representation of real-world objects through domain modeling, utilizing various tools and widgets to enhance usability. After establishing a domain model, users can easily switch to scenario modeling mode to prune the model for scenario generation.

User Interface of ODME

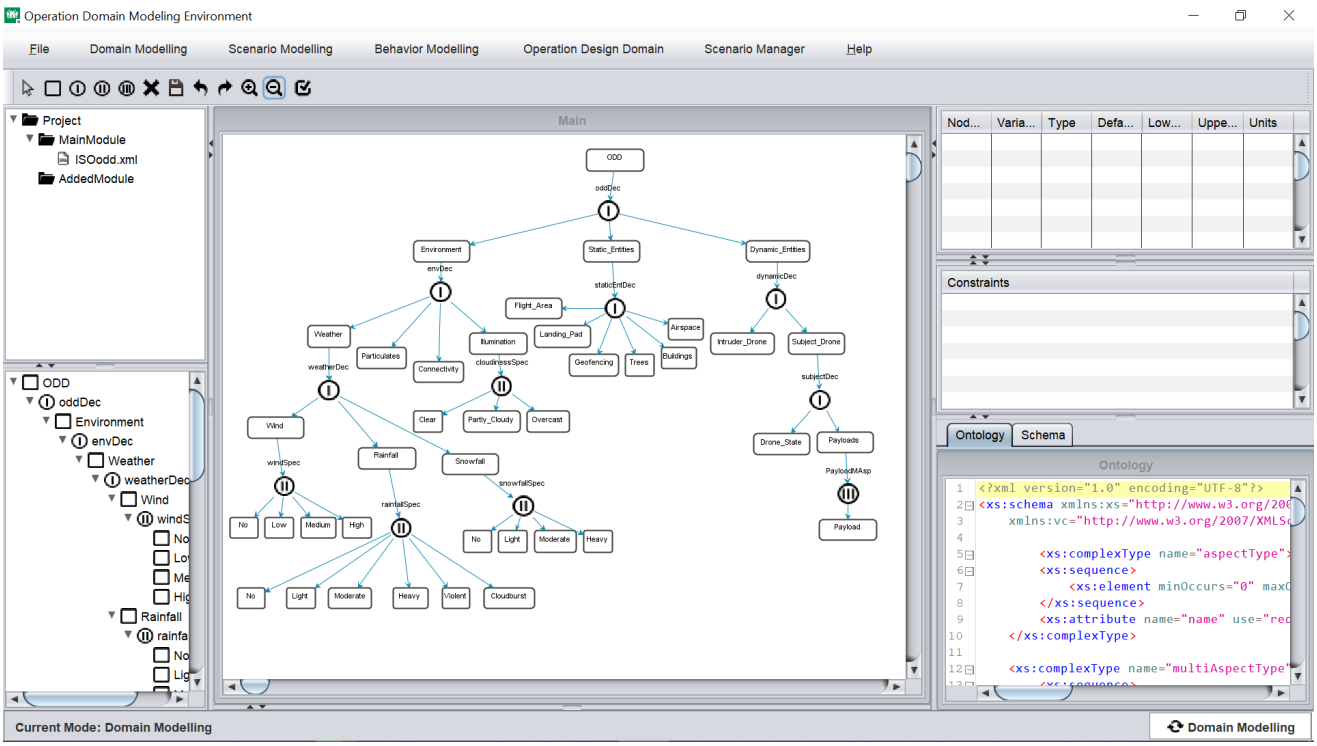

Figure 2.10: ODME User Interface

The User Interface (UI) of ODME is designed to provide the facility to create and edit the system components easily. Some of the key features include:

- **Toolbox:** This provides vital tools for the building and fine-tuning of the SES model.

- **Project Window:** It represents the project structure in a tree-like manner.
- **Drawing Panel:** A user interface used to develop the SES model.
- **Synchronized Tree Window:** It visually reflects changes made to the model. Validation and Import/Export Options: This assures model integrity and allows data exchange.

ODME is implemented in Java SE, utilizing libraries like Xerces and JGraphX for XML parsing and graphical data representation.

2.8 Protobuf

Google originally developed Protobuf to solve scalability challenges caused by the high volume of requests and responses between index servers. The protocol employs binary encoding, significantly reducing the size of serialized data. While native support is available for Java, C++, and Python, third-party libraries extend compatibility to additional programming languages [13]. Also, Protobuf allows for serialization and deserialization of data structures by replacing large text-based formats with compact binary representations.

Advantages of Using Protobuf

When used within systems dealing with large scenario datasets, Protobuf provides the following benefits:

Compact Encoding

Protobuf eliminates the overhead of XML's tag-based and UTF-8 structure, thereby signifi-cantly reducing data size [13]. In the work of Gligorić et al. [12], even the short-tag XML format was found to be roughly three times larger than its Protobuf counterpart, highlighting the significantly more efficient data transfer afforded by Protobuf.ta transfer afforded by Protobuf.

Faster Parsing and Serialization

XML parsers perform extra checks for malformed data, which increases overhead in embedded systems [11]. Protobuf integrates data structures during the compile stage of .proto files, speeding up both serialization and parsing [13]. Gligorić et al. [12] also demonstrated that Protobuf outperforms XML in terms of raw parsing performance.

Less Network Traffic and Cost

The same study by Gligorić et al. [12], daily data transmission per device dropped from 2,449,680 bytes (XML) to 662,400 bytes Protobuf, thus reducing both operational costs and network load.

3 Model Driven Development of Operation Design Domain

This section presents an approach on how to optimize complex models within the framework of MDD, whose basic idea is to execute autonomous or semi-autonomous processes, how to integrate them with ODD and how to prepare them for the automation process and how to prepare the outputs of this process for autonomous processes. First, the basic structure and components of the MDD presented in the study will be introduced, and how the essential elements in this process are combined and worked will be explained. Then, the reasons for using PPES, the benefits of using it as a middleware, the techniques and their justifications, the integration of PES and ODD, the automation preparation processes, and how an optimized approach can be obtained with alternative data output approaches will be discussed. The second section will show how this approach can be used in practice through the Drone Field Model, which will be presented as an example implemented on ODME.

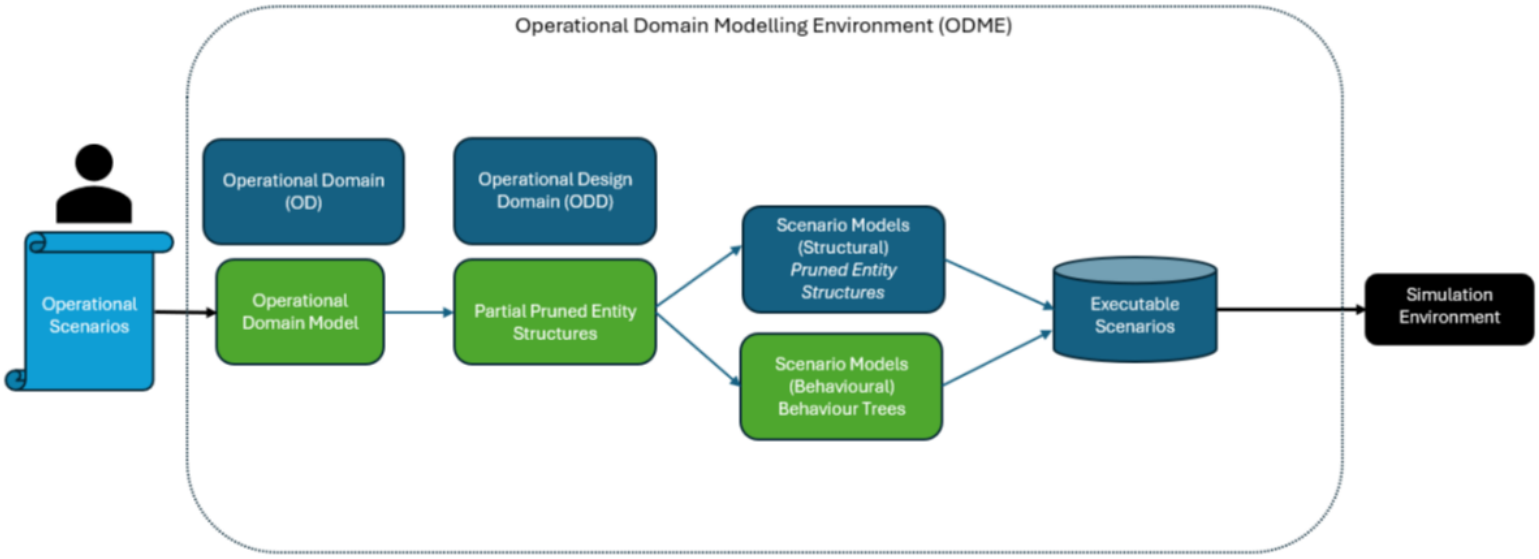

Figure 3.1: MDD Approach

Figure 3.1 shows the structuring of the operational design within the MDD framework. Initially, operational scenarios represent real-world situations that the system will encounter. Then, OD defines general environmental conditions, and ODD defines specific operational conditions. The Operational Domain Model integrates this information and provides the structural conditions necessary for scenarios to fit the real world. Scenario models are divided into two main categories: structural pruned entity structures and behavioral trees; the first category addresses static relationships, while the second covers dynamic processes. Finally, executable scenarios are created from these components and prepared for testing in a simulation environment.

PPES is a component of the MDD process. The following sections will mention its features and how they contribute to the modeling process.

3.1 Partial Pruning Entity Structure

PPES acts as an "intermediate layer" before the generation of scenarios in MDD processes and prepares the structure in which ODD and scenario data are processed together. ODD defines within which environmental, geographical, or performance limits the system can operate; however, these definitions alone do not fully reveal the detailed use cases turned into scenarios. This is where PPES comes into play:

Managing the Operational Domain Model:

- The domain model is like a "proof of concept" that defines the general environmental and functional conditions under which the system can operate.

- PPES breaks down these broad definitions into clearer and more manageable parameters, determining which conditions are critical and which parameters are essential.

- In this way, the environmental and performance information defined in the ODD provides a clear framework for the scenarios to be used within which boundaries should be varied.

PPES Partial Pruning Logic:

- "Pruning" is the process of eliminating redundant or invalid elements in SES (Sys-

tem Entity Structure) and reaching a customized model.

• Partial pruning, on the other hand, reduces all parameters to certain constraints or ranges instead of completely eliminating them. This is important in terms of showing with which variations the conditions in the ODD can be reused.

The Intermediate Layer That Prepares Scenarios:

• PPES transfers the general data coming from the domain model to the Scenario Model layer in a suitable manner.

• During this time, the intervals determined by the ODD in response to the question "Which values are safe?" are made suitable for the scenario by PPES, and unnecessary or repeated parameters are pruned.

• Thus, a flow is formed as "Operational Domain Model → PPES → Pruned Entity Structure (PES) → Scenario Model".

Automation and Optimized Process:

• The PPES layer systematically handles the ODD and the domain model and prepares them for autonomous scenario generation.

• Thus, unnecessary manual interventions are reduced, and data sets that comply with the boundaries in the ODD are automatically distributed to the scenarios.

• At this stage, the user needs to determine which parameters are critical, which ranges will be tested, and approve the partial pruning rules created by PPES.

Data Consistency and Ease of Integration:

• Since PPES enables the transitional use of ODD parameters in different scenario variants, it increases both data consistency and comparability between scenarios.

• On the other hand, operating the PPES layer before scenario modeling allows easier integration of both the domain model (including the ODD) and the output scenarios.

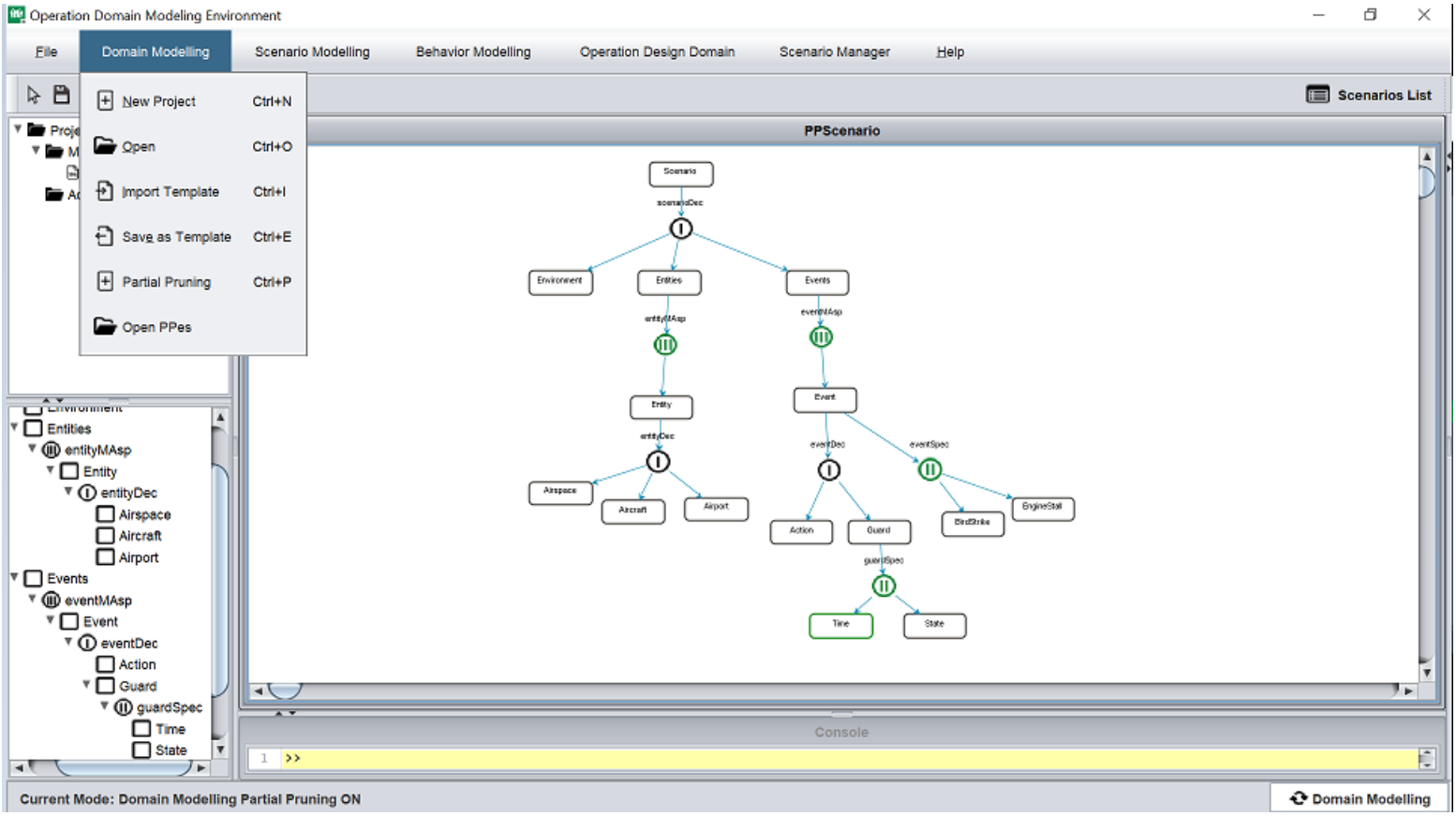

Figure 3.2: Partial Pruning Mode Activation in ODME

Figure 3.2 visually represents how PPES is activated on the ODME.

There are features to make these specifications: Multi-aspect Partial Pruning, Specialization Partial Pruning, Entity Variable Partial Pruning.

In this section, Features will be explained one by one:

3.1.1 Multi Aspect Partial Pruning

Multi-Aspect partial pruning is applied to transform the wide range of options defined in the SES structure of the MDD process, especially in the ODD framework, into a more manageable and purposeful form. The possible scenarios or operational parameters defined in the ODD are often not possible or useful to be fully expressed in a single model, so it is necessary to reach practical models by defining certain constraints or ranges according to the intended use. Below, we explain the multi-aspect partial pruning

process step by step:

Multi-Aspect Ranges

• ODD provides a general framework of environmental, performance, or geographic parameters to be studied.

• Multi-Aspect partial pruning addresses the different aspects within this framework by narrowing them down to a specific range or subset rather than deleting them completely.

Model Becoming More Specific

• The "entity range constraints" mentioned in Figure 3.3 determine which elements or layers the model will generate scenarios on.

• Users decide which ranges the entities will be created in based on the safe or testable conditions in the ODD.

• Thus, general definitions theoretically are limited to ranges close to reality in practice.

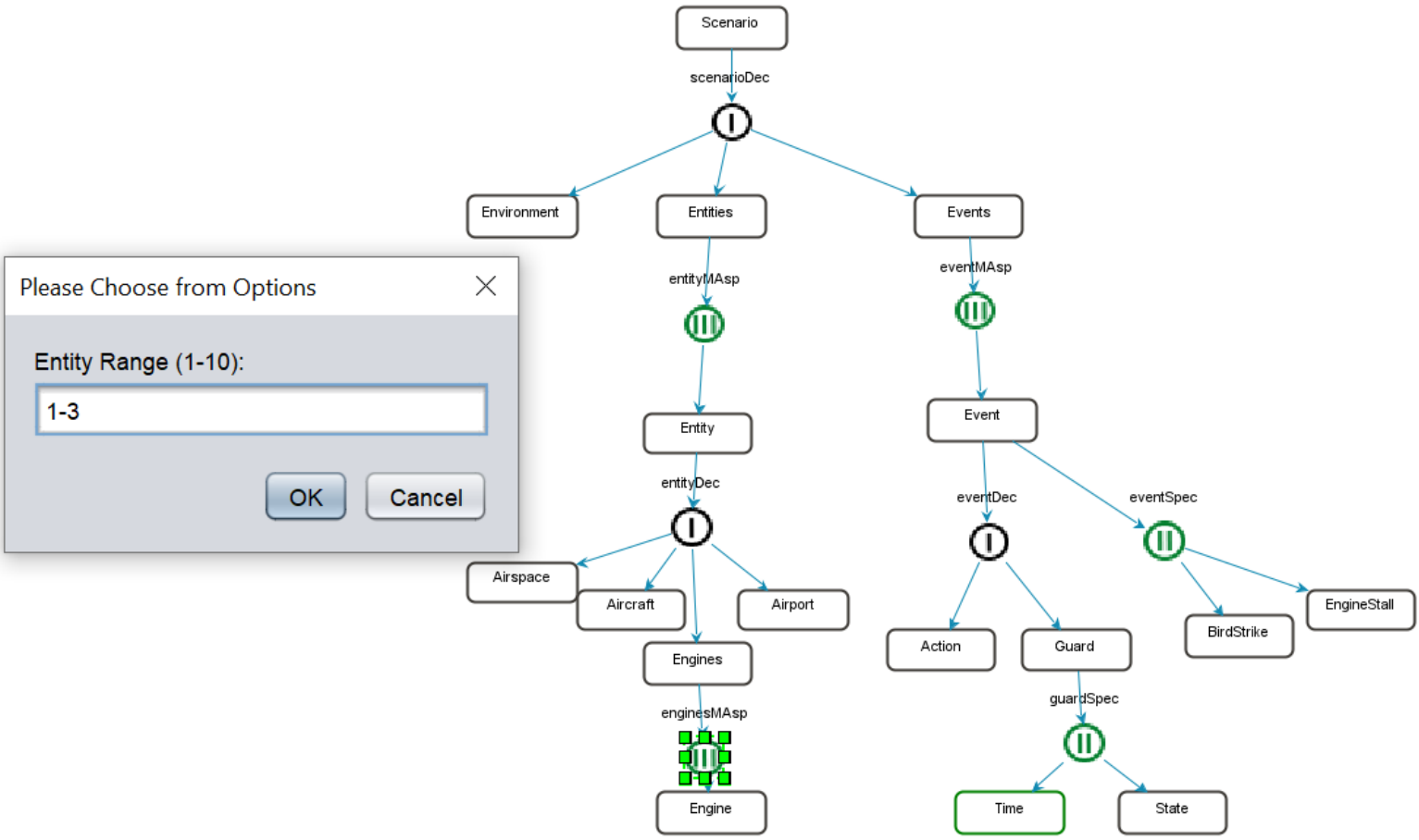

Figure 3.3: Applying Entity Range Limitations to Multi-Aspect Nodes

Application of Entity Range Restrictions and Direction Nodes

- Multi-Aspect nodes can host entities from more than one option or "direction" within a single structure.

- Partial pruning allows the creation of a certain number of entities or a certain range of entities, rather than removing these nodes completely.

- The selection of "user-defined limits" or "specific combinations" seen in Figure 3.3 determines which entities will remain and which will be pruned

Structure After Pruning: PPES Output

- PPES hosts the new state of the Multi-Aspect nodes after this pruning process.

- As shown in Figure 3.4, the entities created for each direction are now reduced to a narrower set of options.

- In this way, options that are theoretically possible in ODD but may be unnecessary or overly broad in practice are systematically filtered.

Automation-Supported Scenario Preparation

- In scenario production within the scope of the MDD approach, it is important to obtain models that are compatible with ODD but also suitable for automation.

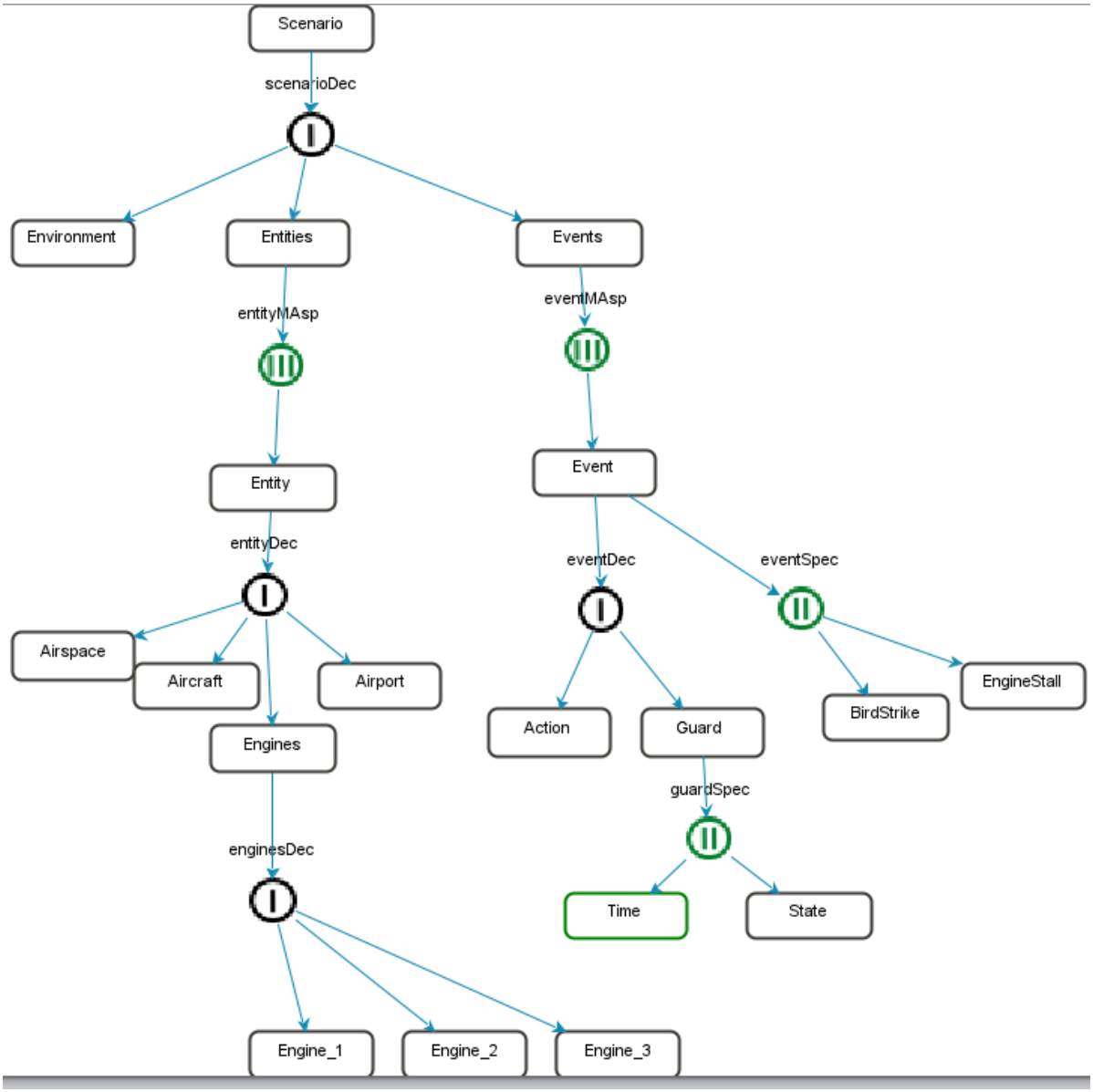

Figure 3.4: Result of Multi-Aspect Partial Pruning Process

3.1.2 Specialization Partial Pruning

Specialization Partial Pruning aims to manage the complexity of specialization nodes in the SES involved in the MDD process. In this case, unnecessary nodes need to be systematically pruned.

- Pruning is the process of removing unselected or unnecessary nodes without disrupting the entire model.

• Instead of deleting everything, it preserves the necessary nodes according to expert opinion or ODD requirements and cleans the redundant ones.

• Thus, the model becomes more focused, both preserving the ODD boundaries and preventing excessive growth.

Consistency and Correctness of the Model • After pruning the relevant nodes, the remaining model variants are reviewed to see if they still meet the requirements of the ODD.

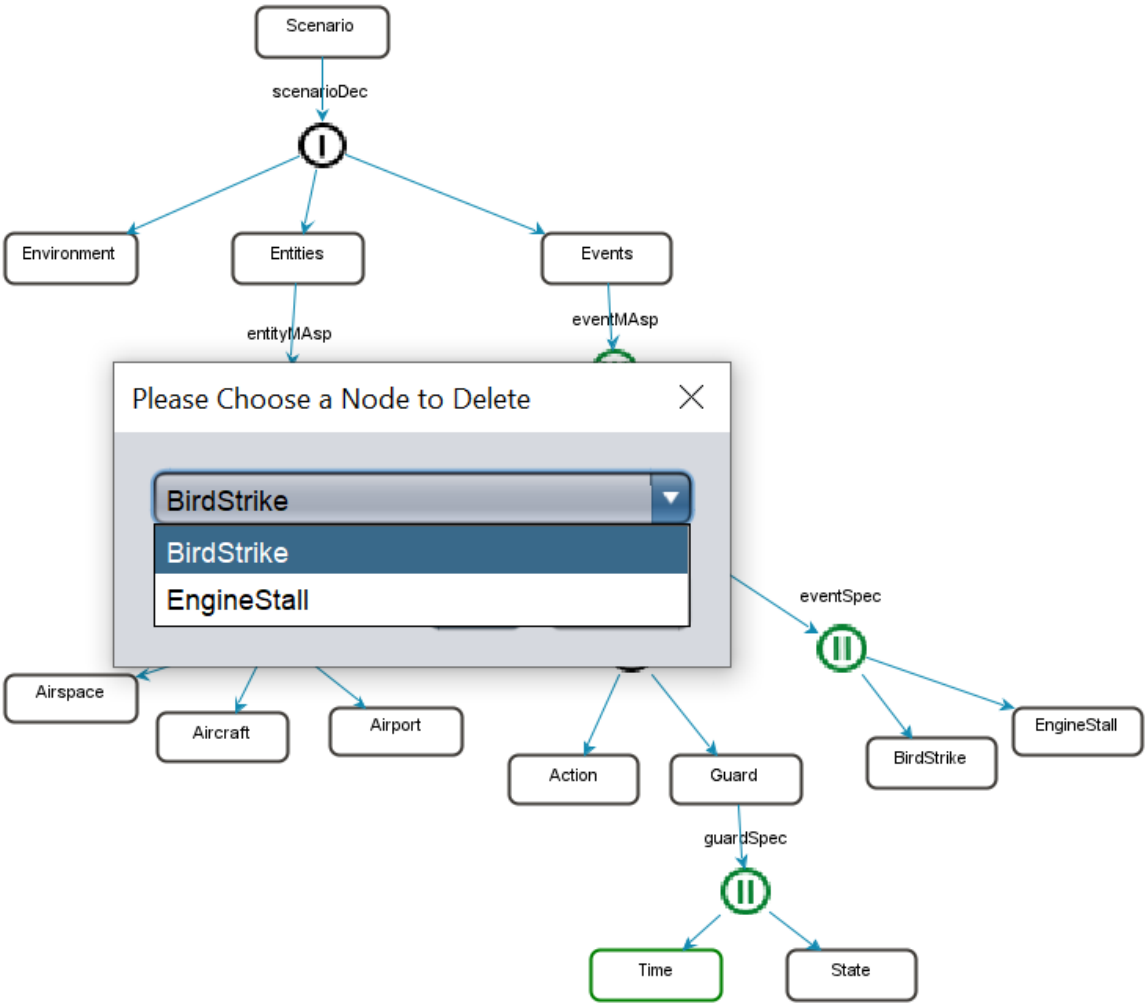

Figure 3.5: Specialization Partial Pruning Process

Figure 3.5 demonstrates the process of specialization partial pruning in detail. It

shows how selected nodes are identified and pruned from the model, with the interface providing tools to confirm and validate these changes.

3.1.3 Entity Variable Partial Pruning

The Entity Variable Editing Partial Pruning feature adjusts entity variables within specified lower and upper boundaries. Boundaries define the permissible range a variable can assume, ensuring that the model behavior remains realistic and within expected parameters. This avoids domain models that are physically or logically impossible, which could otherwise lead to erroneous analysis. By varying these boundaries, users can integrate different operating conditions.

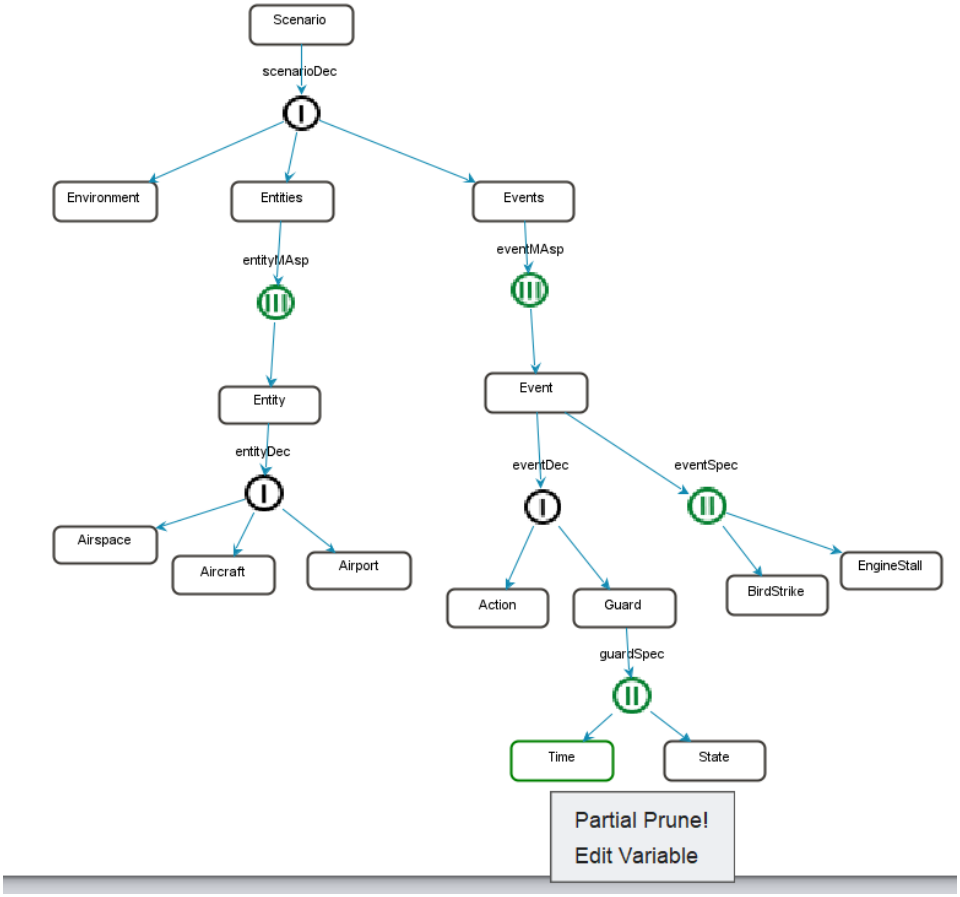

Figure 3.6: Accessing Entity Variable Partial Pruning

Figure 3.6 shows how the Entity Variable Partial Pruning interface is accessed within the modeling environment.

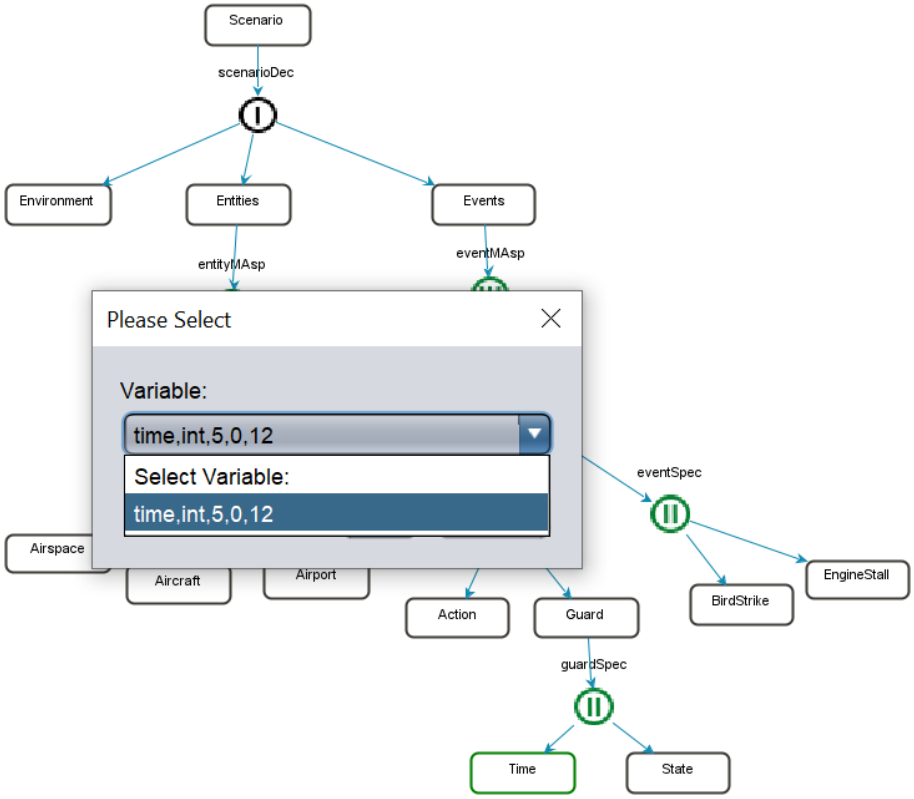

Figure 3.7: Selection of Entity Variables for Partial Pruning

Figure 3.7 demonstrates the selection of specific entity variables within the Variable Editing Partial Pruning interface. Once selected, the variable becomes active and ready for boundary adjustments, which is the next step in fine-tuning the model.

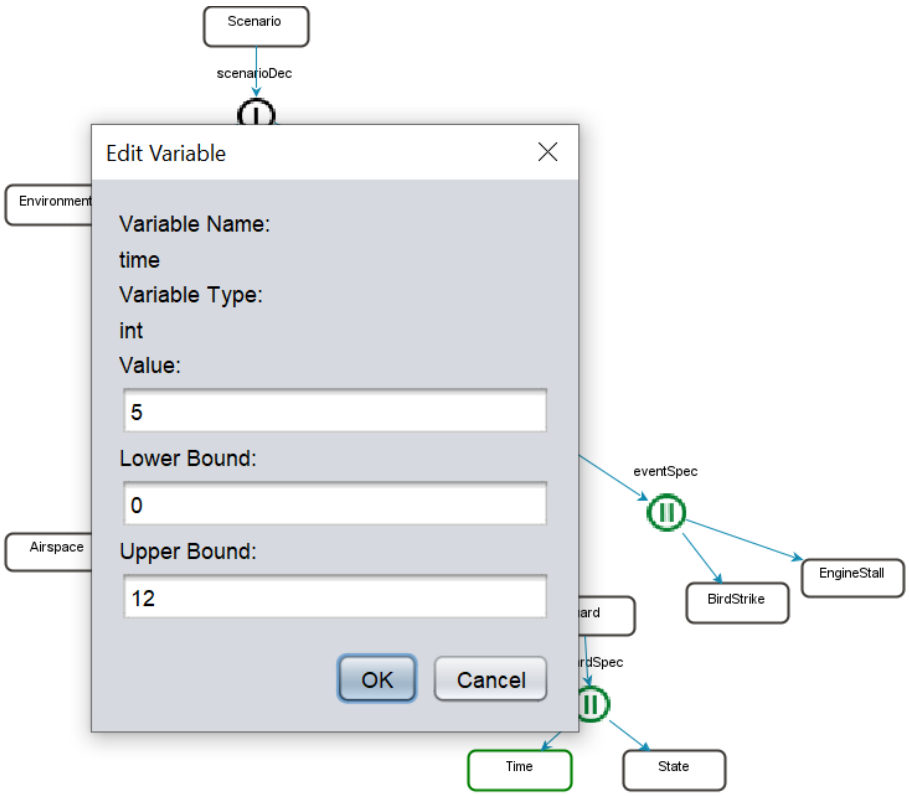

Figure 3.8: Setting Boundaries for Selected Entities Variable

Figure 3.8 shows the process of setting the boundaries for the selected variable. The interface provides various tools such as sliders, input fields, or graphical representations—that allow users to adjust the lower and upper bounds for the variable. These boundaries define the acceptable range for the variable, ensuring that the model remains realistic and within expected parameters.

These features allow us to convert the domain model into a specific model according to needs. This is developed at the domain model stage and can be integrated with the ODD Manager.

Torens et al. states that ”The ODD Manager enables ODD to be exported in XML or YAML format, allowing for further computations. Additionally, it displays and lists all models in the table and ODD list, providing users with a comprehensive perspective.’ [30].

3.1.4 Integration of PPES with ODD

The ODD Manager module is an ODME feature designed to integrate ODD boundaries. Ensure data assurance in scenario modeling processes.

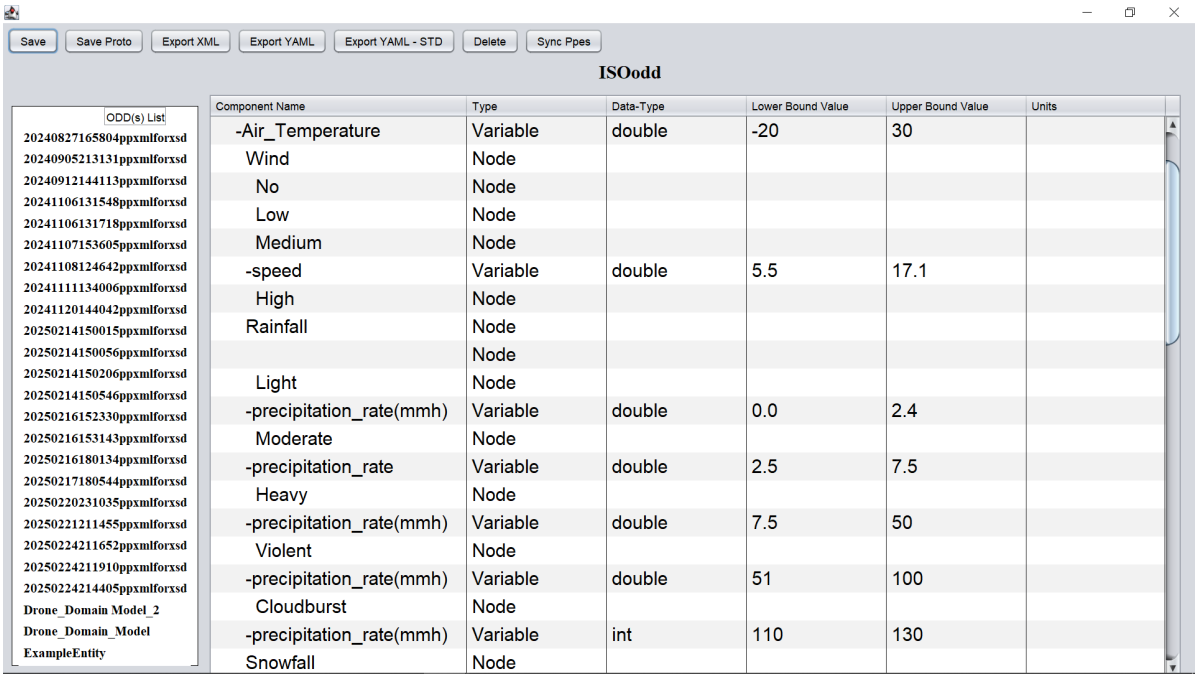

Figure 3.9: ODD Manager

PPES integration with ODD enables ODD to be included in MDD processes. With this integration, ODD becomes a data visualization feature and an active component in scenario generation monitoring for safety assurance and specification of current industry standards. PPES directly integrates the parameters and constraints ODD defines into modeling entities and scenarios.

PPES Synchronization in the ODD Manager module accurately manages PPES files. The ”Sync PPES” button centralizes file repositories and automatically synchronizes them to reduce manual effort.

Since PPES performs the necessary specific pruning as a role before including ODD in the domain model process, the optimization process continues, and all data can be monitored in the ODD manager, and the necessary output operations can be taken.

ODD Manager allows users to export edited PPES files in both XML and YAML formats.

3.1.5 Alternative Data Management for ODD Manager

Using text-based markup language in domain model designs may cause integrity and large-scale storage issues related to the size of the file and the location where it will be kept. Protobuf solves these issues.

A .proto file section A.1, central to the Protobuf serialization process, acts as a data-structure schema.

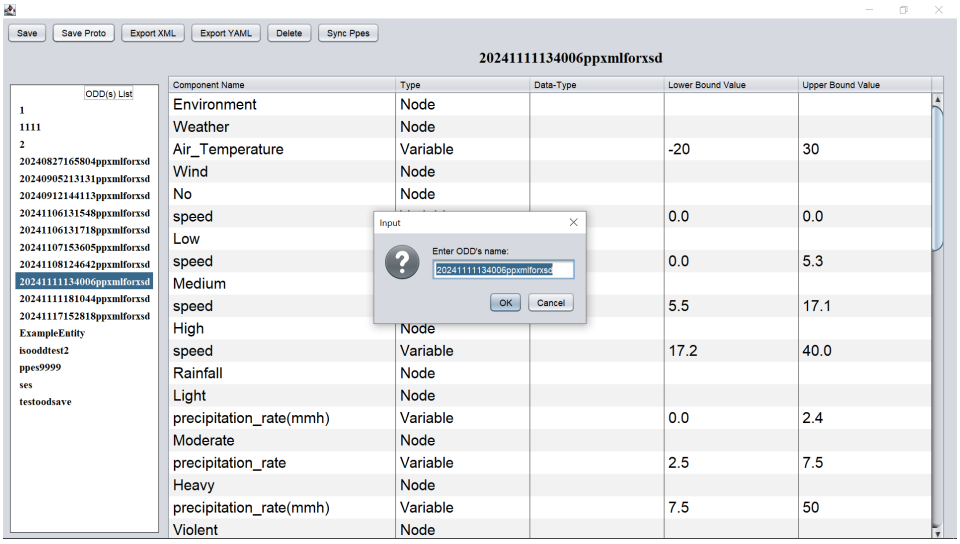

Figure 3.10: Saving ODD to Proto

Figure 3.10 shows how to save edited PPES files after integration with ODD and highlights the feedback provided to users during the saving process.

Once the edits are completed, the entire set of data within these files undergo serialization into.bin files using Protobuf. This binary format is compact and conducive to fast data processing, making it ideal for handling large scale datasets typical of complex simulations.

By employing a binary format, Protobuf reduces data size compared to traditional text-based formats.

3.1.6 Additional PPES Features to ODME

Current ODME-produced domain models were produced in a single place, and a new model had to be created for the changes made. To overcome this, PPES creates a PPES file specific to each model and saves each PPES in its own named file in XML format. Each XML file is given a unique name with a timestamp so that all PPES outputs can be separated from each other.

The PPES Saving Folder Structure feature ensures that model data are stored systematically, addressing challenges such as version control conflicts, file misplacement, and difficulties in traceability.

The folder structure employs a hierarchical design, allowing users to store files in clearly defined categories. Each folder corresponds to a specific criterion relevant to the modeling project.

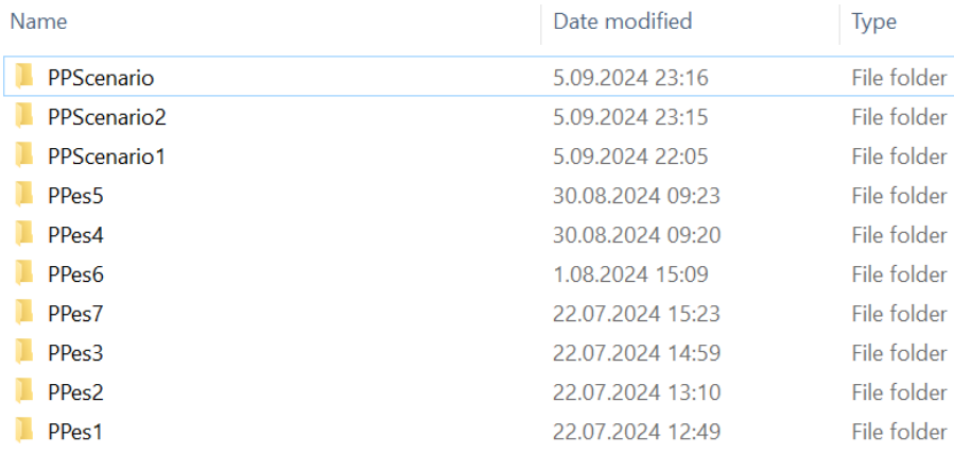

Figure 3.11: Process of Naming and Storing Files with Timestamps

Figure 3.11 demonstrates the process of naming and storing files. The naming convention integrates unique timestamps, which include the exact date and time of file creation or modification. These timestamps act as unique identifiers, preventing confusion between files with similar names and providing a historical context for changes made to the model.

Including timestamps enhances traceability, ensuring that every model iteration can be tracked throughout the development process. This approach minimizes errors and

facilitates collaboration among users.

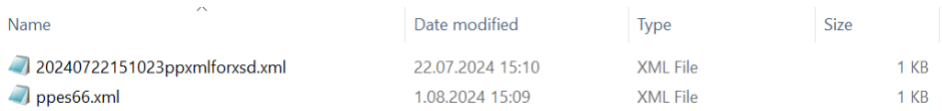

Figure 3.12: Final Structured Folder Containing Saved PPES Files

Figure 3.12 displays the final structured folder containing saved PPES files. It highlights the consistency of file naming, the inclusion of timestamps, and the organization into subfolders, reflecting the logical categorization described above.

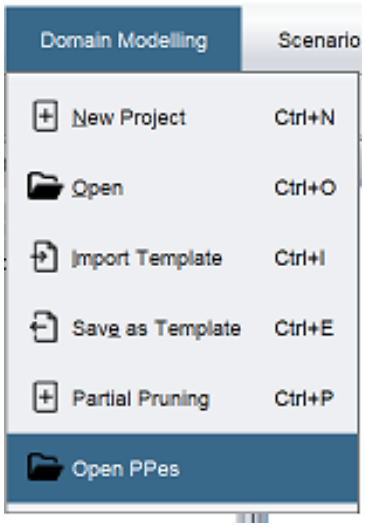

Figure 3.13: Accessing the 'Open PPES' Feature within ODME

Figure 3.13 shows how the "Open PPES" function is accessed within the ODME. This helps users navigate seamlessly within the ODME platform to reach the PPES files user generate variants of previous PPES models.

Once an XML file is selected, the "Open PPES" functionality supports a streamlined editing process, allowing users to modify and improve without interruption.

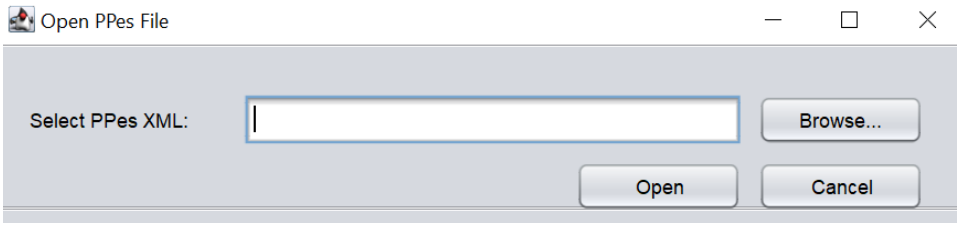

Figure 3.14: Procedure for Selecting a PPES XML File

Figure 3.14 illustrates the procedure for selecting a PPES XML file. It visually represents the steps in choosing the file from the folder structure. The user selects the desired XML file, which is then loaded into the editing interface, ready for modification. The interface's layout is designed for clarity, with relevant sections of the XML file presented and organized for ease of editing.

After selecting and partial pruning an XML file, assigning a new name to the modified file is a critical step. This process ensures clarity and prevents confusion, especially when working with multiple model iterations.

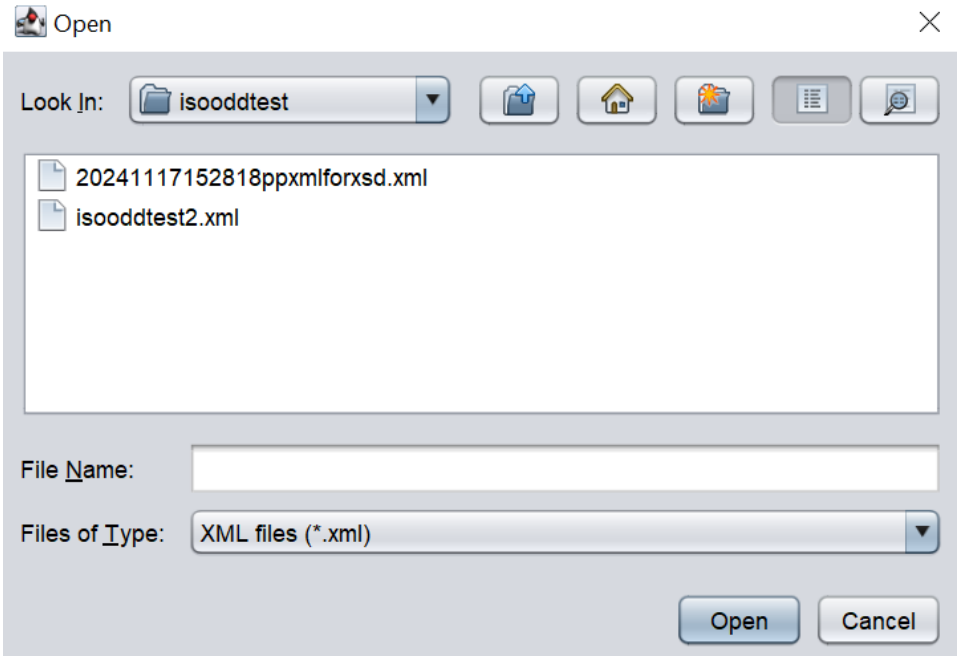

Figure 3.15: New Naming Post-Modification of PPES Files

After making necessary adjustments and improvements to the file, users are prompted to assign a new name to the modified version.

Figure 3.15 highlights the interface features that guide the user in naming the file appropriately, ensuring that different file versions can be tracked effectively.

3.2 Case Study

In this section, we will examine the technical implementation of the Drone domain model developed using ODME in the context of aviation simulations.

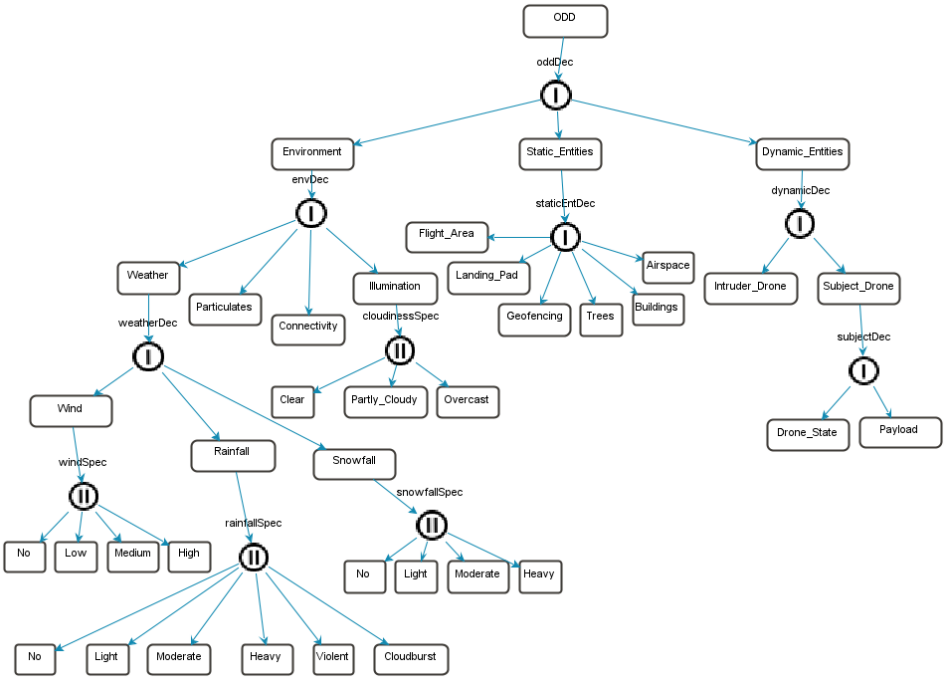

Figure 3.16: Drone Domain Model

The Drone domain model is structured to include detailed descriptions of environmental conditions, such as weather (e.g., wind intensity, rainfall, snowfall) and visual factors like illumination, which directly impact system performance. It also defines static entities (e.g., flight areas, landing pads, trees, buildings) that act as obstacles and dynamic entities (e.g., drones), which interact with the system during the simulation. These dynamic entities are further detailed with specific parameters such as operational status and payload configurations, ensuring high fidelity in behavior modeling.

In addition to the definition of entities, the drone domain model incorporates constraints and operational parameters that guide the simulation under varying environmental conditions. For example, it allows the system to respond to high wind speeds or low-visibility scenarios with precise adjustments. By structuring the file in this way, the Drone domain model ensures alignment between simulated conditions and real-world operational challenges.

PPES

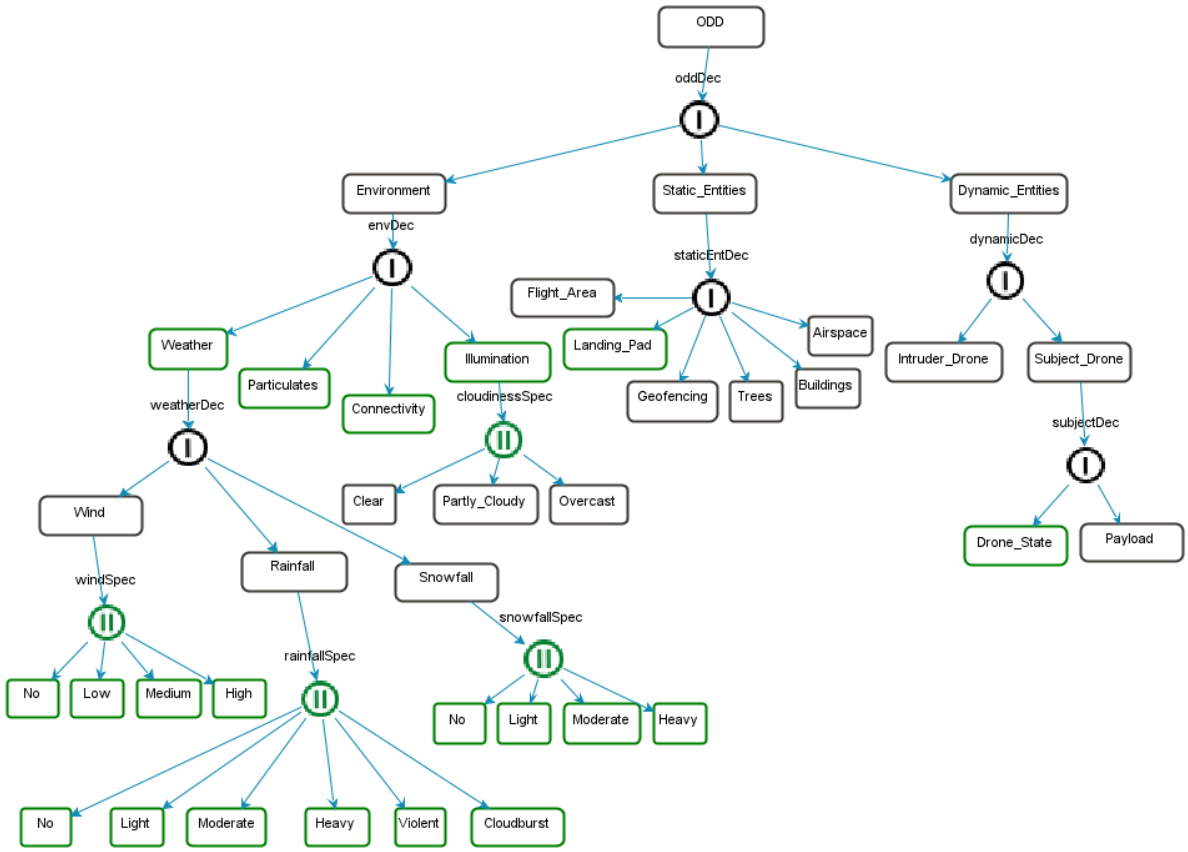

Figure 3.17: PPES Interface During the Pruning Process

Figure 3.17 illustrates the PPES interface during the pruning process within the Operational Design Modeling Environment. This interface showcases how the system visually represents the selection, elimination, or specification of nodes and entities within the

model.

Specialization Partial Pruning

Specialization pruning is applied to optimize the model by removing the 'Violent' entity in rainfall specialization.

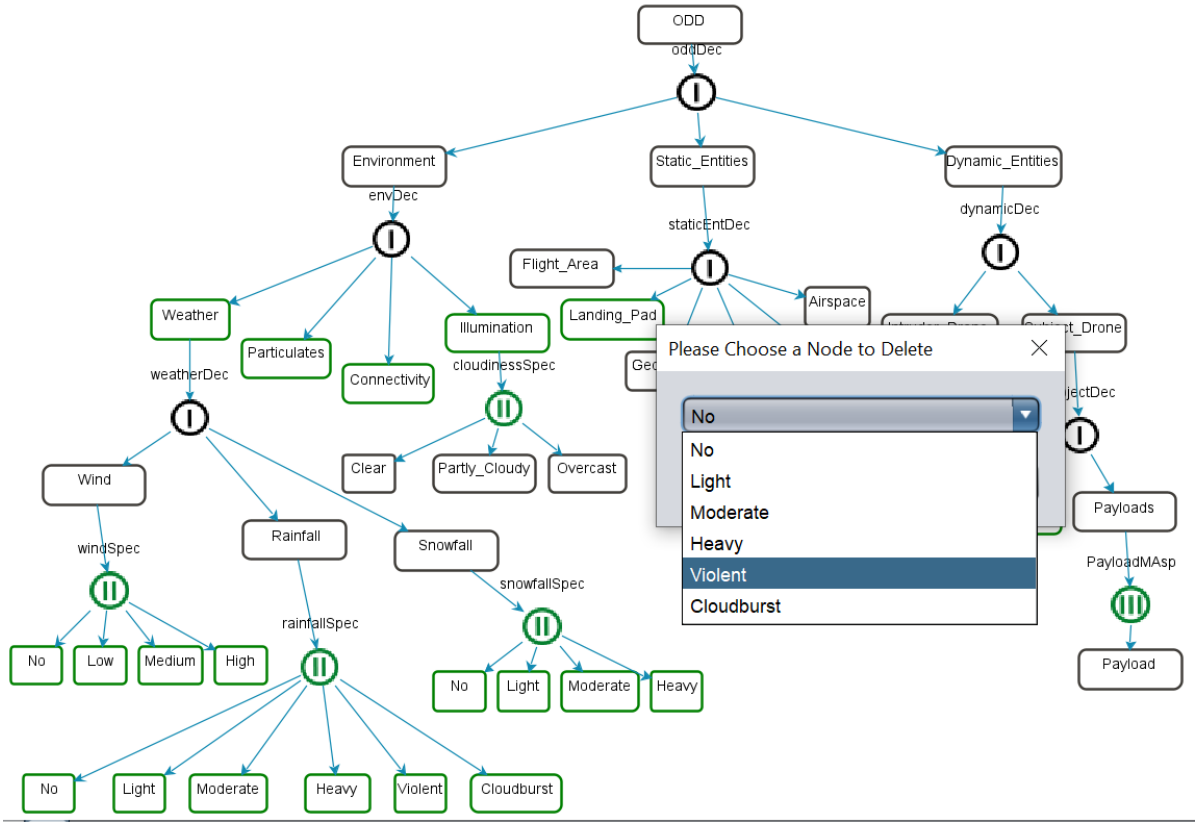

Figure 3.18: Specialization Partial Pruning

During domain modeling, it was identified that excessive rainfall characterized as "Violent" rarely occurred in actual operational data. Retaining "Violent" rain introduced a redundant that did not align well with the system's main testing objectives, thus increasing overhead without offering meaningful new domain models. Environment domain model included multiple rainfall categories (e.g., Light, Medium, Heavy, Violent). The "Violent" sub-entity was flagged for possible removal after reviewing scenario requirements, constraints, and actual observed rainfall intensities.

Navigated to the rainfall specialization node. Reviewed the sub-entities (Light, Me-

dium, Heavy, Violent) under “Rainfall.” Selected “Violent” in the interface for potential removal, as it did not contribute to a realistic or needed scenario range.Confirmed that removing “Violent” would not interfere with or misapply references from any other nodes. Ensured no critical test scenario required that level of rainfall intensity.

In Figure 3.18, “Violent” was explicitly pruned, which effectively simplified the model’s rainfall categories to Light, Medium, and Heavy. This action was committed and updated in the domain model, so the model tree no longer displayed “Violent.”

By removing an not required specialization node, the refinement process streamlined the scenario set while retaining critical rainfall categories. With realistic rainfall intensity levels, test scenarios were aligned closer to actual operational constraints, improving clarity in scenario generation. It also allows all possible domain model variations to be made with the remaining categories.

Entity Variable Partial Pruning

In the Entity Variable Partial Pruning within the Cloudburst entity, variable editing partial pruning was applied to adjust the lower and upper bound values of the precipitation variable.

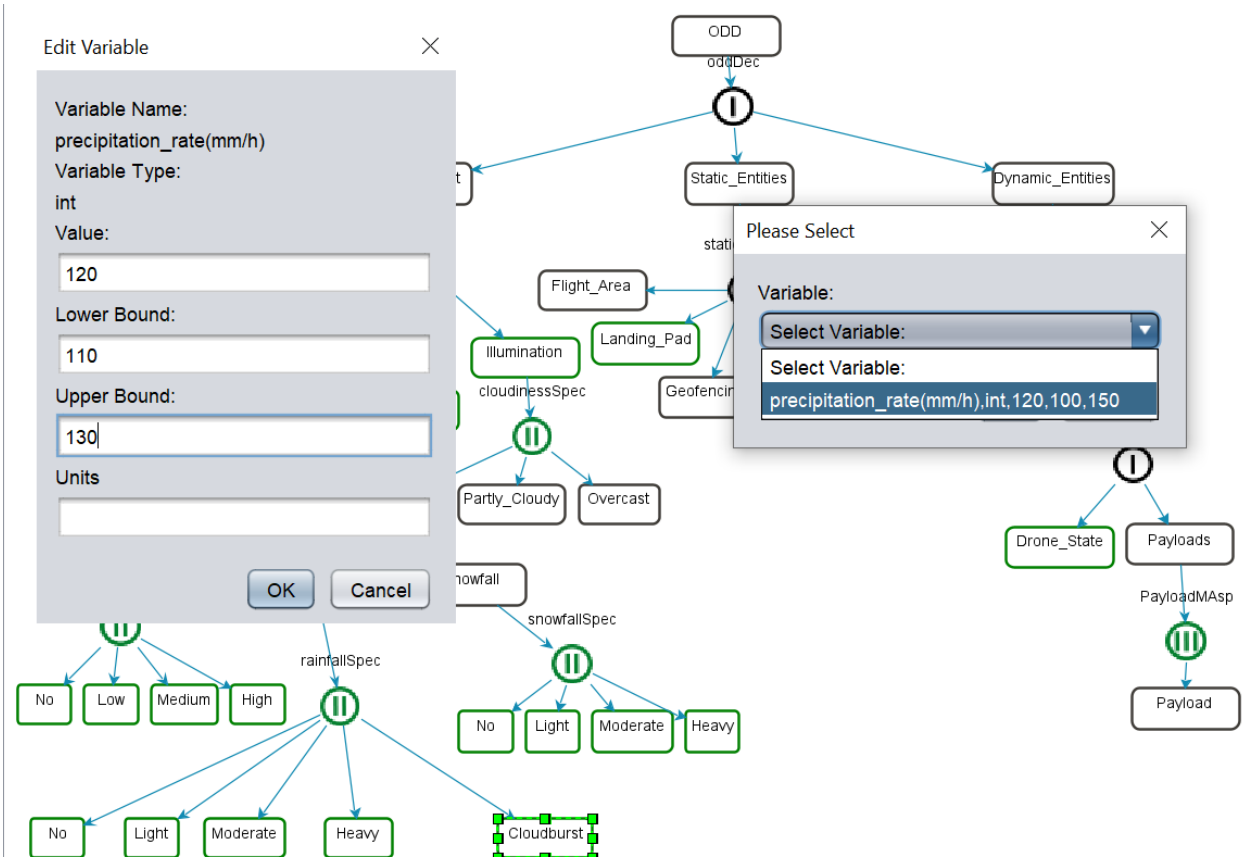

Figure 3.19: Entity Variable Partial Pruning

An initial review of real-world precipitation data showed that cloudburst events typically occurred within a narrower precipitation range than initially modeled. Retaining overly broad ranges for precipitation rate risked producing unrealistic or less applicable scenarios, diluting the focus on truly critical rainfall conditions.

Within the 'Cloudburst' entity, the 'precipitation rate' variable was initially set with wide default limits. These extremes needed refinement to reflect operational scenarios in anticipated data.

Accessed ODME's "Entity Variable Partial Pruning" interface for the Cloudburst entity. Reviewed the existing precipitation variable's bounds to determine which segments were not vital for typical cloudburst scenarios. Specified narrower lower and upper limits.

Confirmed no conflicts with other system components. Ensured the new precipitation

rate boundaries still captured realistic high-intensity rainfall events.

In Figure 3.19, the lower and upper boundaries were adjusted in the ODME interface, and the domain model was updated so that the refined precipitation range was used for subsequent scenario generation.

With a closer match to actual cloudburst rates, simulations can better capture the potential operational challenges of intense rainfall. Not required extremes were pruned, focusing the scenario model on meaningful precipitation values. The domain model fosters more credible simulations and test cases by bounding the precipitation variable more accurately.

Multi Aspect Partial Pruning

Multi-aspect partial pruning provides the 'Buildings' entity, effectively breaking it into three distinct buildings. This step illustrates how focusing on specific aspects of a multi-aspect entity can streamline the model, allowing each building to be addressed individually within the domain model.

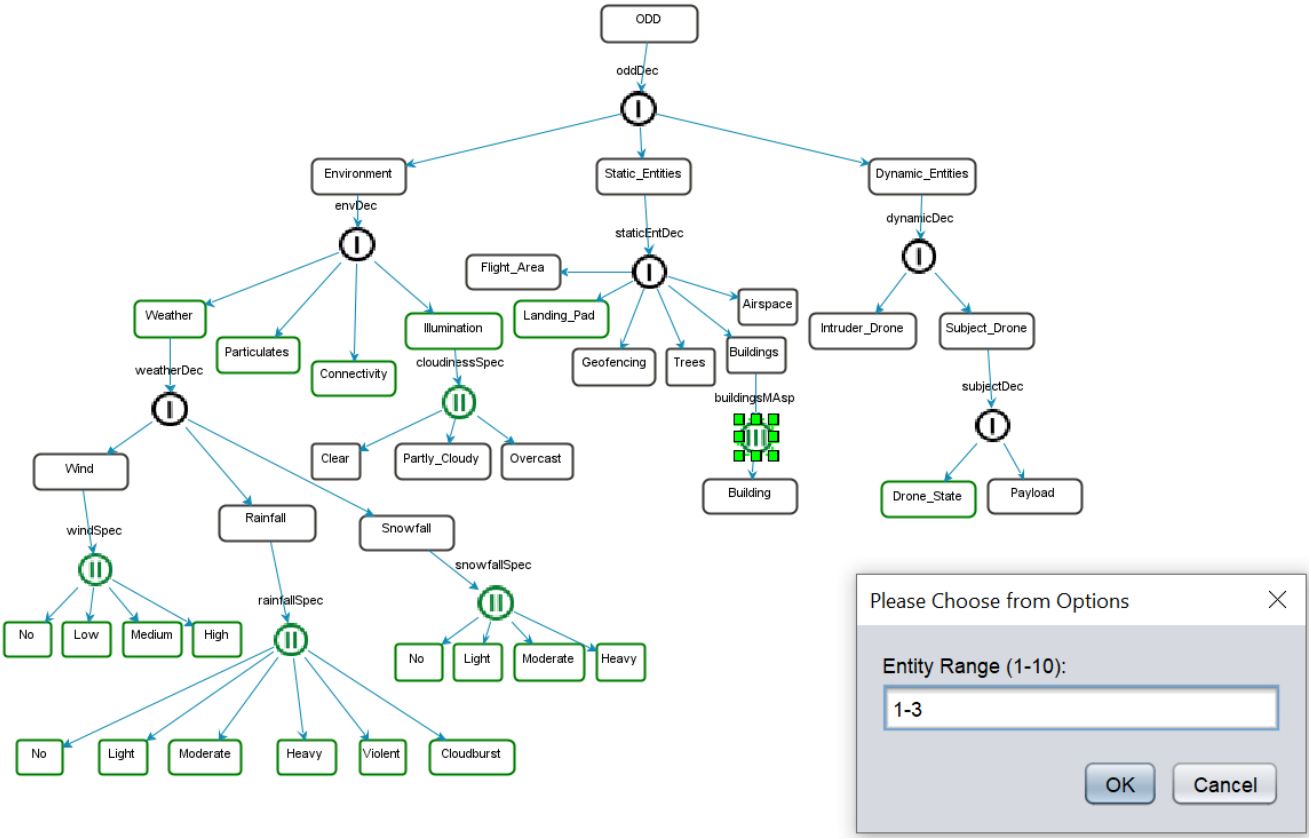

Figure 3.20: Multi Aspect Partial Pruning Implementation

Initially, the domain model might have lumped multiple buildings into a single multi-aspect node under "Buildings," without clear bounds on how many could be generated. Certain scenarios required specifying attributes for separate buildings, which would be cumbersome if all buildings were treated as a single entity.

Under the multi-aspect definition, "Buildings" existed as a repeating structure, but it lacked detailed cardinality or upper limits, allowing for either many buildings or a single entity representing them all in general. Domain analysis revealed a typical use case of three building structures, each potentially with unique entities.

In Figure 3.20, navigate to the multi-aspect node for "Buildings." Three distinct building entities were created to reflect real-world environment constraints, and partial pruning was applied to set an explicit range for generating separated building entities.

Confirmed each new building entity did not conflict with other domain model aspects. Ensured that limiting the count to three buildings aligned with both scenario

requirements and the underlying domain knowledge.

Building1, Building2, and Building3 were defined as separate entities. The domain model now recognizes three distinct building structures, each of which can carry unique attributes or constraints.

Focusing on a known building count reduces model complexity, and each building can be managed independently. Separate building entities allow for more precise scenario definitions, such as differences in building height, layout, or potential obstacles for drones. Scenario testing can systematically handle each building's role in flight paths or potential obstruction, strengthening the simulation's fidelity to real conditions. It also allows all possible domain model variations to be made with the remaining categories.

PPES Save Structure and ODD implementation

The process of saving the PPES structure and implementing the ODD focuses on optimizing the model. This step aims to monitor the safety assurance and adapt current industy standards of the model. Ensure that the structure is efficiently saved after changes, allowing continuous updates and modifications to be shown within the ODD Manager.

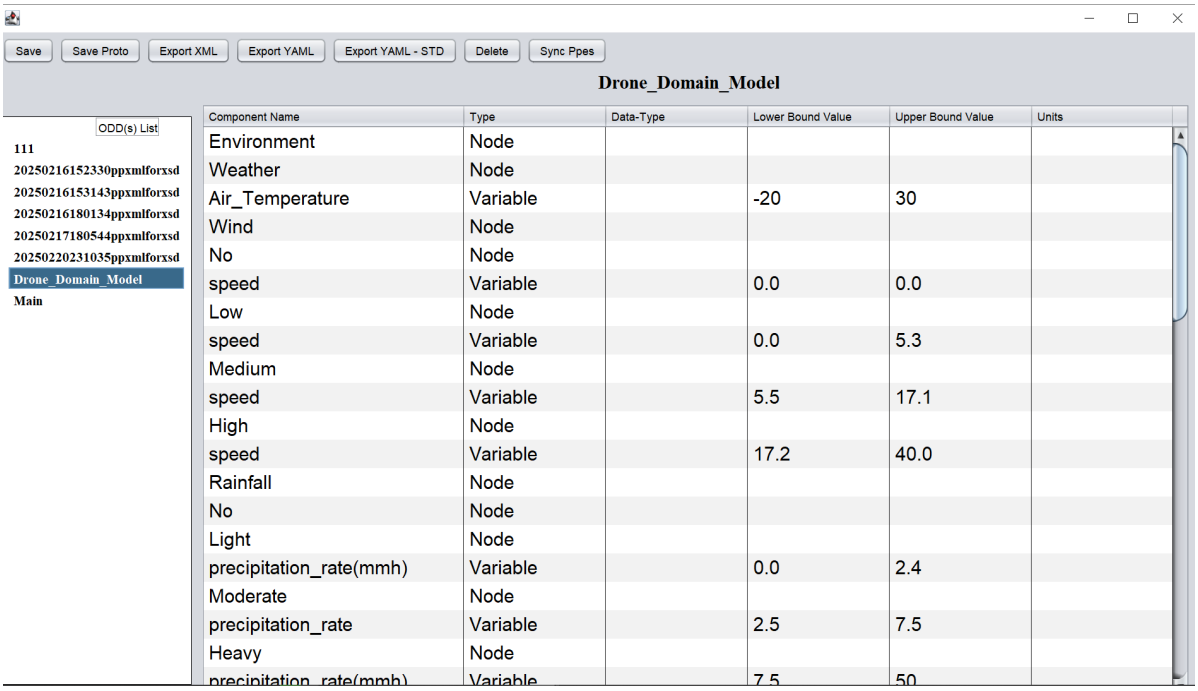

Figure 3.21: PPES Save Structure and ODD implementation

As partial pruning decisions evolve, an updated model of the PPES structure is essential for maintaining a clear lineage of the model. Continuously saving incremental changes ensures that any newly pruned or altered components are properly reflected in both the domain model and the ODD integration layer.

The model remains synchronized with operational restrictions by incorporating ODD constraints whenever the PPES domain model is saved. This ensures that the domain model, partial pruning data, and ODD definitions all point to the same reference state, reducing inconsistencies.

In the Figure 3.21, after making partial pruning modifications (Multi-Aspect, Specialization, or Entity Variable), the user engages the "Save Structure" function in ODME. The newly pruned PPES structure is then serialized—often in XML, YAML, or Protobuf—capturing both domain data and ODD constraints as needed. This updated file (along with timestamps) is placed alongside the existing model repository, ensuring traceability.

Each saved domain model, with its accompanying timestamp, can be saved, permitting rollbacks or comparisons between older and newer partially pruned domain models.

During saving, the system can perform validations to ensure that the partial pruning modifications do not conflict with ODD boundaries.

Storing the PPES structure after each change fosters incremental improvements in model variability rather than waiting for a final large-scale update. User can follow the model's evolution for every variants of domain models and see precisely how ODD constraints were applied at each iteration. By merging partial pruned domain model and ODD constraints in each save operation, ODME and the ODD Manager maintain an up-to-date, synchronized state, enabling reliable scenario generation and thorough industry standards.

Open PPES Feature

Open PPES, access is provided to previously pruned XML files. After the PPES process is performed again, the saving process is performed and this is integrated into ODD Manager.

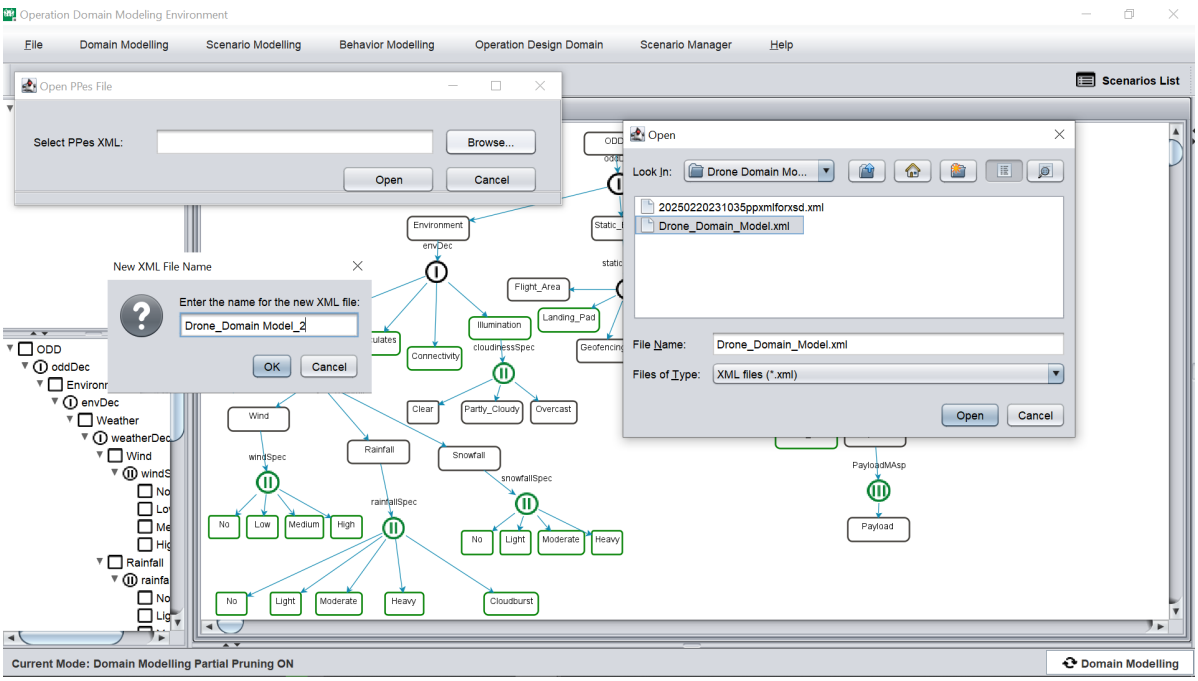

Figure 3.22: Open PPES Feature

Users can create variants of the PPES domain model by opening previously pruned XML files. Re-evaluating previous pruned domain models allows incremental improve-

ments or re-checking of domian models for ongoing alignment with new constraints with user feedback.

In the Figure 3.22 , The user selects the “Open PPES” feature in ODME. A file dialog (or similar interface) appears, listing all XML files created in previous sessions with corresponding timestamps. Once a specific XML file is chosen, ODME loads the partial pruning configuration and displays it in the model tree.

Users may adjust multi-aspect, specialization nodes, or entity variable ranges in light of newly discovered requirements or updated operational data. If new constraints or optimizations are recognized, the user may prune further or restore specific nodes that were previously removed but have become relevant again.

After the loaded model has been refined, changes are saved under unique timestamp, differentiating it from previous domain model files. This updated partial pruning state is then integrated into the ODD Manager, ensuring that safety constraints and operational boundaries remain consistent with the latest modifications. Storing each iteration in XML format makes it easier to exchange partial pruned domain models among users.

ODD Manager Protobuf Feature

In Figure 3.23, Users refine or define the domain model in ODME. Manually created ODD data is also compiled, often describing environment boundaries or safety-critical parameters. The datasets are then mapped or linked together, ensuring coherence.

Once the user validates and finalizes the integrated model, the entire domain model specification is serialized into a proto .bin file. This binary-compatible format, as Protobuf, reduces file sizes compared to text-based alternatives.

Protobuf encoding cuts down on overhead from verbose text tags, leading to more compact files. Reading and writing operations become faster, contributing to improved throughput if data is frequently transferred or reloaded in the workflow. The system supports a streamlined scenario-generation pipeline by retaining PPES details and ODD constraints together in .proto format. Protobuf’s schema-based structure helps maintain data integrity by simplifying the detection of any mismatches.

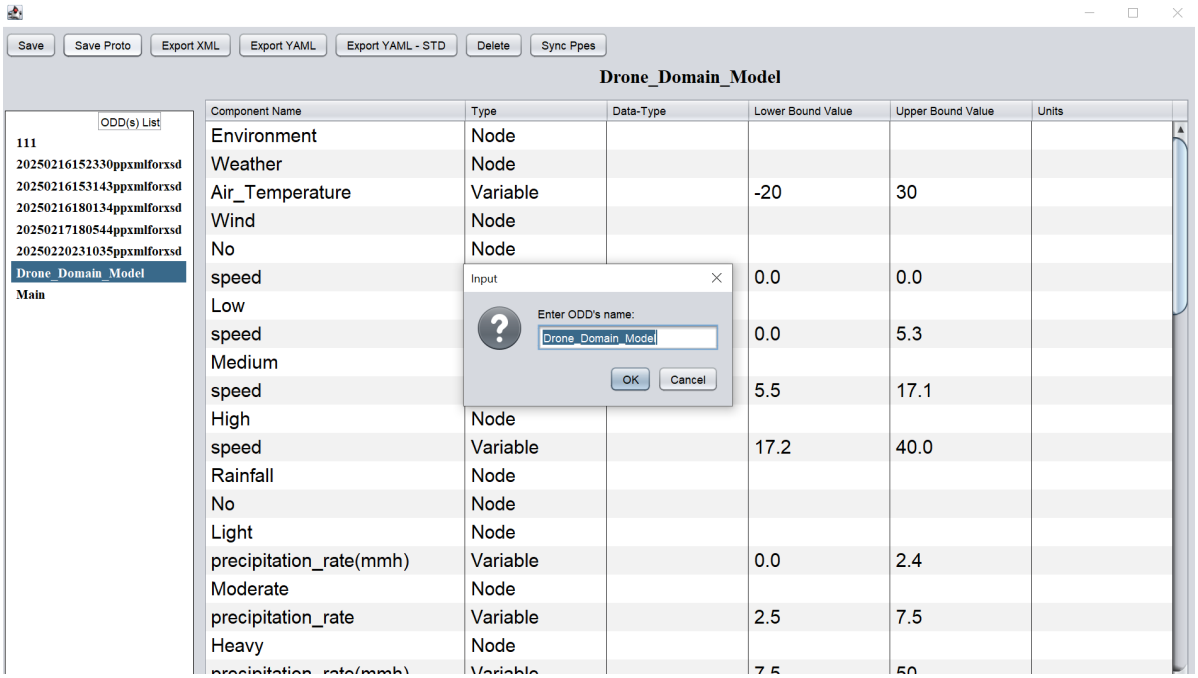

Figure 3.23: ODD Manager Protobuf Feature

4 Findings and Discussion

This section examines the results of implementing the PPES approach in this study. The objective is to highlight the improvements in performance, safety assurance, and data management achieved by incorporating PPES into the MDD process. First, the findings will be presented, followed by a discussion of the existing MDD approach on ODME tool.

4.1 Findings

This section explores the findings from applying the PPES approach to the Drone Domain Model case study. In this study, both performance measurements and observations from the model development processes are assessed together. The domain modeling case study, which utilizes Multi-Aspect Partial Pruning, Specialization Partial Pruning, and Entity Variable Partial Pruning, has increased goal-oriented structure and data size. As the scope of the defined SES structures expands, managing variables becomes more intricate. However, the PPES approach effectively addresses these challenges and provides a more goal-oriented modeling process.

Goal-oriented structure: Within the study's scope, scenario modeling experiments conducted with Multi-Aspect Partial Pruning, Specialization Partial Pruning, and Entity Variable Partial Pruning approaches provided specifications in goal-oriented structure. The PPES approach has made the drone domain model specific. Thus, an important step has been taken toward making complex systems goal-oriented.

When looking at the case study output section A.2, rainfall specialization cloudburst precipitation boundaries were reduced to the range of 110-130, building_1 and building_2 were created, sun position degree boundaries were reduced to the range of 30-60, air temperature boundaries were reduced to the range of 0-10, Heavy and Cloudbrust entities were used in rainfall, and no and light entities were used in snowfall. A goal-oriented model was achieved by making specifications.

Data Size Improvement: Data initially serialized in ODME and kept in XML format

was converted to Protobuf format, decreasing from 767 to 40 bytes for domain modeling layer. This creates a cost and time advantage for transferring and storing scenario data sets.

A powershell script was written section A.3 for the studies conducted with the case study and the file sizes were listed.section A.4

Additionally, when the PES scenario model files are examined with the script in section A.6, the advantage of using Protobuf in PPES over current PES outputs is observed, as file sizes of only 40 bytes are noted, compared to 51110 bytes with same scenario genaration process.section A.7

Traceability and Version Control: With the “save structure” feature of PPES, XML saves of different optimizations can be stored on a timestamp basis. The “Open PPES” feature can quickly reload and edit these variants. This approach makes it possible to track the model’s evolution more easily, intervene quickly in faulty versions, and conduct reliable version control.

Powershell script was written section A.8 to show section A.9the file path. This proves how PPES can develop multiple models with many different versions within the same domain model.

ODD Integration: ODD Manager’s monitoring of PPES model boundaries has guided the user during the domain model phase, providing safety assurance for the scenario generation process.

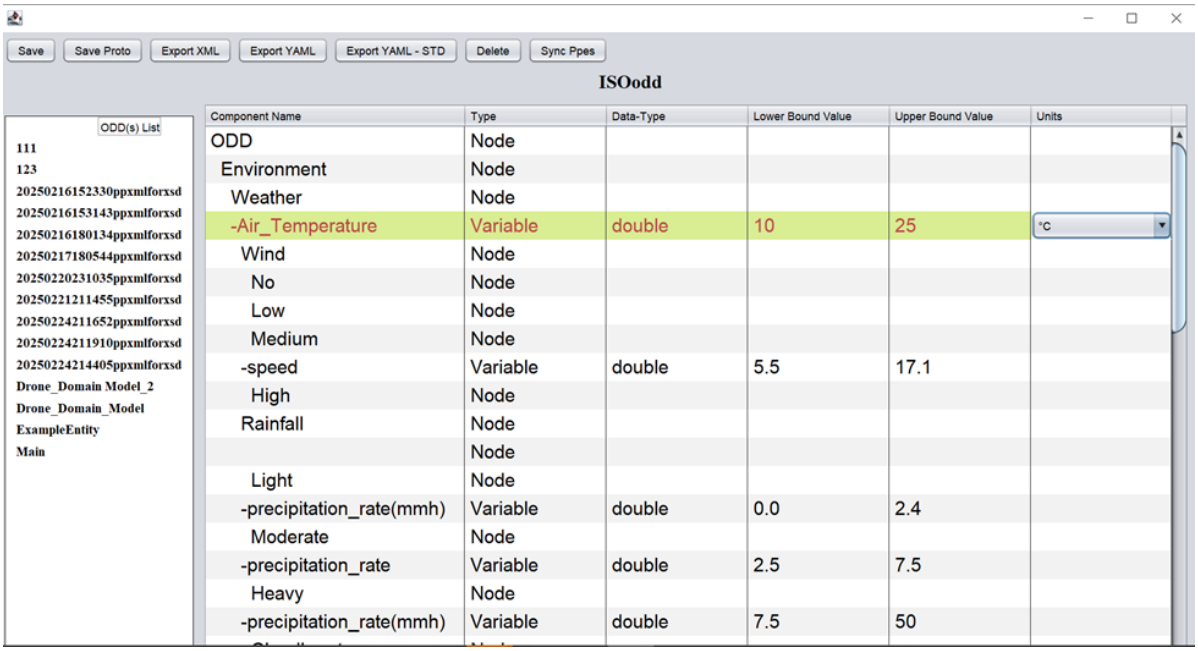

Figure 4.1: ODD Manager

In the figure, PPES and MDD processes can be monitored with the ODD manager.

4.2 Discussion

4.2.1 Interpretation of the Results

The findings from applying the PPES approach in this study highlight several key improvements in MDD processes:

Goal-Oriented Structure and Efficiency: Introducing Multi-Aspect Partial Pruning, Specialization Partial Pruning, and Entity Variable Partial Pruning techniques proved essential in addressing the increasing complexity and data size of expanding SES structures. Instead of pruning all at once (as in a current PES model), PPES maintained the necessary flexibility to accommodate future modifications. As a result, the drone domain model could evolve incrementally, preserving clarity and flexibility in intermediate stages.

Performance Improvements: By converting XML data to Protobuf format in domain modeling layer, the data size decreased from 767 bytes to 40 bytes. Furthermore, an

advantage of 40 bytes, as opposed to 51110 bytes, is observed when Protobuf in PPES is compared to current PES outputs. This significantly lowered costs for transferring and storing large scenario datasets, enhancing both runtime performance and maintainability. It also suggested that integration of data optimization methods can yield substantial benefits in MDD workflows where scenario models are frequently serialized or exchanged.

Improved Traceability and Version Control: The saving structure and "Open PPES" features enabled the preservation of different pruned models, each with a unique timestamp. This structure not only simplified targeted rollbacks to versions but also facilitated the coordination of model variants in same domain model.

Integration of ODD for Safety Assurance: Ensuring operational constraints at the domain model level through ODD integration helped shape realistic and safer scenarios in the design [30]. This approach—instead of adding safety rules afterward—prevented the generation of irrelevant or unsafe variants. In doing so, the system supported a more robust safety assurance pipeline aligned with real-world operational parameters.

Overall, the interpretation of these results confirms that PPES can act as a valuable intermediary step in MDD, bridging domain modeling with final scenario outputs. By combining PPES approach, data optimization, and safety integration, the study points to a more iterative, goal-oriented workflow that mitigates both overhead and guesswork in scenario development.

Comparison with the Current Approach of ODME

The differences between the current approach to ODME and the techniques in this study, the MDD approaches, and the differences in alternative data management are discussed. Important in terms of understanding the evaluation in the study: Although the scenario modeling layer used PES in the current approach, PPES served as an intermediate layer in the study, offering a different perspective.

1. PPES in intermeiate layer between SES and PES:

PES is a structure derived from SES with complete pruning, where not required nodes and entities are pruned, and the final structure for the scenario model.

PPES provides a partial structure instead of a finished pruning result. Instead of pruning, it can be applied at domain modeling layer and leaves the scenario details to be completed in the future partially open.

For example, in the YAML output given figure, the rainfall specialization cloudburst precipitation boundaries were reduced to the range of 110-130, thus no clear scenario model was defined, and all scenario models that could be created within this range were allowed. Thus, if the user or the system needs additional structures/parameters later, they can easily be updated or expanded via PPES.

2. PPES to ODD Integration to MDD Process:

ODD in the current approach: – In the current approach, ODD is used to monitor safety assurance with a decoupled structure.

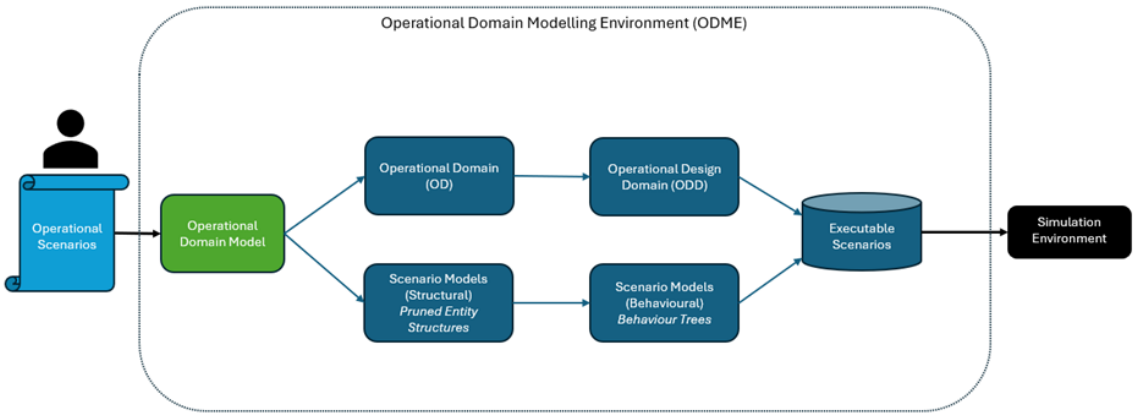

Figure 4.2: Current MDD Approach

The figure shows that ODD has no direct integration with Scenario Models.

ODD Integration in PPES: Figure 3.1 shows, PPES is used as a fundamental part of the MDD process, while ODD serves as a monitor for safety assurance.

3. Entity Variable Pruning:

In the current approach: An entity variable is set to a specific value or a range with

clear boundaries.

PPES: In the YAML output, multiple scenario models are kept in the same domain model by defining ranges rather than making a specific definition of entity variables. This method establishes a flexible structure for users who seek to explore boundary ranges of scenarios. Allows users to define a partial boundaries range for an entity variable. This way, users can retain parameters that are not yet finalized to be expanded. This allows specific variants to be created easily if needed in the future.

4. Multi-Aspect Pruning:

In the current approach: It is done by specifying the number of entities produced, and no explicit upper limit restriction is set; in other words, as many entities as theoretically desired can be generated or pruned under the multi-aspect node.

PPES:
This approach defines an explicit upper limit for the entities of the same multi-aspect node. In addition, cardinality is not determined, lower and upper limits are defined, and entities are generated accordingly. The multi aspect partial pruning in the YAML output proves this.

Thus, flexibility is maintained in the rest of the scenario by creating entities within the specified range, and goal-oriented structure that may arise from generating too many entities is controlled.

5. Specialization Pruning:

In the current approach: In a specialization node, the child entity to be selected is pruned at once and the other options are completely pruned.

PPES: Instead of making the selection all at once, the user or system keeps multiple child entities, only the selected child entity pruned. The specialization partial pruning in the YAML output proves this.

In this way, both flexibilities provided in the specialization node are preserved.

6. Version Control with Save Structure Features In the current approach: When you save a scenario model, that scenario state is fixed. If a new variant is required, users need to generate a new scenario model file.

PPES: PPES Save Structure allows the partially pruned model at each stage to be saved in XML, YAML, or Protobuf format. After the prototype model is edited, it is possible to work on many prototype domain models within the same domain model thanks to the unique timestamp provided during saving, which provides effective filing.

The output obtained with the Powershell script proves this.

Thus, beyond the pruning, partial pruning at different stages can also be kept as versions, reverted, and new models can be generated in the same domain model.

7.Alternative Data Solutions:
In the current approach it allows exporting XML and YAML in limited versions for domain modeling layer.

Additionally, there is no variability option in the outputs obtained from the scenario modeling layer and the file sizes are higher. It is observed that the current PES outputs measure 51110 bytes in section A.7.

This study optimizes data storage by offering protobuf opportunities and exporting block flow YAML format.

With the proof provided by the Powershell script, Protobuf provided the size advantage and integrity of the data sets.

4.2.2 Limitations

On ODME, the PPES layer still requires user intervention, which can create limitations in scenario generation without expert knowledge.

Although the transition to the Protobuf format has seen a significant size reduction, it is moving away from the textual (human-readable) format; this may require platform-dependent solutions in the version control or debugging phases.

Whenever the standard output format in the ODD Manager is modified, the proto structure must be revised accordingly to maintain alignment with the new specifications.

Although Protobuf is utilized, the XML format remains a necessary component within ODME.

5 Conclusion and Future Work

This chapter presents the conclusion of the study and discusses future work.

5.1 Conclusion

Manual validation processes, which include real-world tests and physical verification of system outputs, are impractical. Similarly, the aviation industry faces extensive testing requirements that exceed its current capabilities. Integrating AI into aviation systems is transforming the industry through safety and automation. Regulations set by authorities such as the EASA are important for the safe integration of AI-based systems, and simulations are supported to ensure safety compliance.

This research aims to streamline and optimize the scenario generation process within MDD for AI-based aviation systems. By introducing the PPES as an intermediary layer, the study seeks to integrate ODD, thereby enhancing model accuracy and parameter coverage. Additionally, it focuses on improving data output by Protobuf to ensure data integrity and reduce storage requirements, preparing for automated scenario generation.

How can scenario generation process be optimized through a methodological approach that can resolve the decoupled structure of ODD within the MDD process by determining operational parameters, thereby preparing for an automated scenario generation process in AI-based aviation systems?

How can the data output of scenario models be optimized to effectively manage size while ensuring integrity and consistency for the preparation of automated scenario generation process?

Above the research questions are answered thorough the methodology presented in this research work, the thesis introduces the PPES as an intermediate layer within the MDD process. This approach integrates ODD constraints directly into the scenario generation framework, enhancing scenario diversity evaluation and ensuring parameter coverage. The study employs the PPES to refine model components, eliminate redund-

ant parameters, and facilitate the tracking of model variants with version control. By converting data outputs from text-based to binary format using Protobuf, the methodology addresses data size and integrity issues, leading to preparing automated scenario generation process.

This research contributes to the field by introducing the PPES to enhance scenario generation in MDD for AI-based aviation systems. By integrating ODD and optimizing data outputs with Protobuf. These innovations preparation of the automated scenario generation processes.

5.2 Future Work

In future work, the main aim will be to integrate the automation of the scenario generation process into the MDD within ODME. This step will be crucial for reducing manual intervention. A key aspect of this will focus on enabling end-to-end automation of the MDD process, such as the integration of CARLA or Unreal Engine GUIs, by ensuring that every stage, from the operational model to scenario execution, is fully automated.

References

[1] *Artificial Intelligence Roadmap: A human-centric approach to AI in Aviation*. Tech. rep. URL https://www.easa.europa.eu/en/downloads/109668/en. EASA, Feb. 2020.

[2] Uwe Aßmann, Stefan Zschaler and Gerd Wagner. "Ontologies, meta-models, and the model-driven paradigm". In: (2006).

[3] BSI Group. *PAS 1883:2018 - Assuring the safety of automated vehicle trials and testing - Code of Practice*. Accessed 12 April 2023. 2018. URL: `https://www.bsigroup.com/en-GB/automotive/publications/PAS-1883-2018/`.

[4] B. Chandra Karmokar et al. "Tools for scenario development using system entity structures". In: *AIAA Scitech 2019 Forum*. 2019, p. 1712.

[5] I. Colwell. "Runtime restriction of the operational design domain: A safety concept for automated vehicles". MA thesis. University of Waterloo, 2018.

[6] *Concepts of Design Assurance for Neural Networks (CoDANN)*. Tech. rep. URL https://www.easa.europa.eu/en/downloads/112151/en. EASA and Daedalean AG, Mar. 2020.

[7] *Concepts of Design Assurance for Neural Networks (CoDANN) II*. Tech. rep. URL https://www.easa.europa.eu/en/downloads/128161/en. EASA and Daedalean AG, May 2021.

[8] Umut Durak and Tuncer Ören. "Towards an ontology for simulation systems engineering". In: *Proceedings of the 49th Annual Simulation Symposium*. Apr. 2016, pp. 1–8.

[9] Umut Durak et al. "Scenario development: A model-driven engineering perspective". In: *2014 4th International Conference On Simulation And Modeling Methodologies, Technologies And Applications (SIMULTECH)*. IEEE. 2014, pp. 117–124.

[10] EASA. *EASA Concept Paper: First usable guidance for Level 1 machine learning applications*. Issue 01. 2021.

[11] R. Engelen. “Code generation techniques for developing lightweight XML Web services for embedded devices”. In: *ACM symposium on Applied computing*. New York: ACM Press, 2004, pp. 854–861.

[12] N. Gligorić, I. Dejanović and S. Krčo. “Performance evaluation of compact binary XML representation for constrained devices”. In: *2011 International Conference on Distributed Computing in Sensor Systems and Workshops (DCOSS)*. IEEE, 2011, pp. 1–5.

[13] Google Protocol Buffers. *Google’s Data Interchange Format, Documentation and open source release*. `http://code.google.com/p/protobuf/`.

[14] EuroCAE WG-114/SAE G-34 Artificial Intelligence Working Group. *Artificial Intelligence in Aeronautical Systems: Statement of Concerns*. Tech. rep. URL https://doi.org/10.4271/AIR6988. SAE International, Apr. 2021. DOI: `10.4271/AIR6988`.

[15] S. Gupta, J. Sprockhoff and U. Durak. “Scenario Coverage of Operational Design Domain for AI-Based Systems in Aviation”. In: *AIAA SCITECH 2025 Forum*. 2025, p. 2326.

[16] S. Hallerbach. “Simulation-based testing of cooperative and automated vehicles”. PhD thesis. Universität Oldenburg, 2020.

[17] ISO. *ISO 21448:2022 Road vehicles — Safety of the intended functionality*. 2022.

[18] ISO. *ISO 34503:2023 Road vehicles, test scenarios for automated driving systems specification for operational design domain*. 2023.

[19] F. Kaakai et al. “Toward a Machine Learning Development Lifecycle for Product Certification and Approval in Aviation”. In: *SAE International Journal of Aerospace* 15.01-15-02-0009 (2022). DOI: `10.4271/01-15-02-0009`.

[20] R. Kashyap. “Artificial intelligence systems in aviation”. In: *Cases on modern computer systems in aviation*. IGI Global, 2019, pp. 1–26.

[21] T.-G. Kim et al. “System entity structuring and model base management”. In: *IEEE Transactions on Systems, Man, and Cybernetics* 20.5 (1990), pp. 1013–1024.

[22] F. Klück et al. “Using ontologies for test suites generation for automated and autonomous driving functions”. In: (Oct. 2018), pp. 118–123.

[23] A. Naja et al. “Towards a Scenario Toolkit for Autonomous Systems”. In: *Simulation Notes Europe (Simul. Notes Eur.)* 34.1 (2024), pp. 13–21.

[24] Tuncer I Ören and Bernard P Zeigler. “System theoretic foundations of modeling and simulation: a historic perspective and the legacy of A Wayne Wymore”. In: *Simulation* 88.9 (2012), pp. 1033–1046.

[25] Jerzy W Rozenblit and Bernard P Zeigler. “Representing and constructing system specifications using the system entity structure concepts”. In: *Proceedings of the 25th conference on Winter simulation*. 1993, pp. 604–611.

[26] SAE International. “Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles”. In: *SAE International* 4970.724 (2018), pp. 1–5.

[27] J. Sprockhoff et al. “Model-Based Systems Engineering for AI-Based Systems”. In: *AIAA SCITECH 2023 Forum*. 2023, p. 2587.

[28] C. Sun. “Operational Design Domain Monitoring and Augmentation for Autonomous Driving”. PhD thesis. 2022.

[29] O. Topçu et al. “Model driven engineering”. In: *Distributed Simulation: A Model Driven Engineering Approach*. Springer, 2016, pp. 23–38.

[30] C. Torens et al. “From Operational Design Domain to Runtime Monitoring of AI-Based Aviation Systems”. In: *2024 AIAA DATC/IEEE 43rd Digital Avionics Systems Conference (DASC)*. IEEE. 2024, pp. 1–9.

[31] Susan B. Van Hemel, Jacqueline MacMillan and Gregg L. Zacharias, eds. *Behavioral Modeling and Simulation: From Individuals to Societies*. National Academies Press, 2008.

[32] H. Weber et al. “A framework for definition of logical scenarios for safety assurance of automated driving”. In: *Traffic Injury Prevention* 20.sup1 (2019), S65–S70.

[33] DEEL Certification Workgroup. “Machine Learning in Certified Systems”. In: (2021). URL https://doi.org/10.48550/arXiv.2103.10529. DOI: 10.48550/arXiv.2103.10529.

[34] B. Zeigler. *Multifacetted modelling and discrete event simulation*. Academic Press, 1984.

[35] B. P. Zeigler and P. E. Hammonds. *Modeling and simulation-based data engineering: introducing pragmatics into ontologies for net-centric information exchange.* Elsevier, 2007.

A Appendix

A.1 Proto

In order to integrate Protobuf, the .proto file should be coded as follows to include the components of the domain model:

Listing A.1: Proto Integration Script

```
syntax = "proto3";

package specialization;

message Var {
    string var_name = 1;
    string lower = 2;
    string upper = 3;
}

message Entity {
    string name = 1;
    repeated Var vars = 2;
}

message Specialization {
    repeated Entity entities = 1;
}

message Root {
    repeated Specialization specializations = 1;
}
```

A.2 Case Study Output

Listing A.2: YAML Domain Model Output

```
ODD:
  Environment:
    Weather:
      Air_Temperature: double
      -Air_Temperature: [0 .. 10] (°C)
        Wind:
          No:
          Low:
          Medium:
        speed: double
        -speed: [5.5 .. 17.1]
          High:
        Rainfall:
          Heavy:
        precipitation_rate(mmh): double
        -precipitation_rate(mmh): [7.5 .. 50]
          Cloudburst:
        precipitation_rate(mmh): int
        -precipitation_rate(mmh): [110 .. 130]
        Snowfall:
          No:
          Light:
    Particulates:
      Intensity: int
      -Intensity: [0 .. 100]
      Size: int
      -Size: [0 .. 10]
    Illumination:
      Luminosity: int
      -Luminosity: [0 .. 12000]
      sun_position(degrees): int
      -sun_position(degrees): [30 .. 60]
        Clear:
        Partly_Cloudy:
        Overcast:
    Connectivity:
```

```
  Static_Entities:
    Flight_Area:
    Landing_Pad:
      area(m2): double
      -area(m2): [1 .. 20]
      Geofencing:
    Trees:
    Buildings:
      Building_1:
      Building_2:
      Building_3:
    Airspace:
  Dynamic_Entities:
    Intruder_Drone:
    Subject_Drone:
      Drone_State:
       altitude(m): double
       -altitude(m): [23 .. 48]
       speed(ms): double
       -speed(ms): [0 .. 10]
       angle: double
       -angle: [-10.0 .. 10.0]
        Payload:
```

A.3 PowerShell Script to specifty data size

Listing A.3: PowerShell Script to specifty data size

```
$oddBinPath = "...\ODME-main\odd.bin"
$ppesFolderPath = "...\ODME-main\Main\PPes"
$oddFolderPath = "...\ODME-main\odd"

$oddBinSize = (Get-Item $oddBinPath).Length

$ppesTotalSize = (Get-ChildItem -Path $ppesFolderPath -Recurse -File |
    Measure-Object -Property Length -Sum).Sum
```

```
$oddTotalSize = (Get-ChildItem -Path $oddFolderPath -Recurse -File |
    Measure-Object -Property Length -Sum).Sum

$xmlYamlTotalSize = $ppesTotalSize + $oddTotalSize

Write-Output "odd.bin File Size: $oddBinSize bytes"
Write-Output "Total Size of XML Output: $xmlYamlTotalSize bytes"
```

A.4 PowerShell Output of Comparison Data Size

Listing A.4: PowerShell Output of Comparison Data Size

```
PS .\\> ...\Untitled1.ps1
odd.bin File Size: 40 bytes
Total Size of XML Output: 767 bytes
```

A.5 PowerShell Output of Comparison Data Size

Listing A.5: PowerShell Output of Comparison Data Size

```
odd.bin File Size: 40 bytes
Total Size of PES outputs: 51110 bytes
```

A.6 PowerShell Script to specifty data size

Listing A.6: PowerShell Script to specifty data size

```
$oddBinPath = "...\ODME-main\odd.bin"
$pesFolderPath = "...\ISOodd"

$pesTotalSize = (Get-ChildItem -Path $ppesFolderPath -Recurse -File |
    Measure-Object -Property Length -Sum).Sum
```

```
Write-Output "odd.bin File Size: $oddBinSize bytes"
Write-Output "Total Size of PES outputs: $pesTotalSize bytes"
```

A.7 PowerShell Output of Comparison Data Size

Listing A.7: PowerShell Output of Comparison Data Size

```
odd.bin File Size: 40 bytes
Total Size of PES outputs: 51110 bytes
```

A.8 show PPES file structure

Listing A.8: PowerShell script to show PPES file structure

```
$ppesFolderPath = "...\ODME-main\Main\PPes"

function Show-DirectoryTree {
    param (
        [string]$Path,
        [string]$Indent = ""
    )
    $folder = Get-Item -Path $Path
    Write-Output "  $Indent [$($folder.Name)]"
    $items = Get-ChildItem -Path $Path
    foreach ($item in $items) {
        if ($item.PSIsContainer) {

            Show-DirectoryTree -Path $item.FullName -Indent " $Indent "
        } else {

            Write-Output "  $Indent $($item.Name)"
        }
    }
}
Show-DirectoryTree -Path $ppesFolderPath
```

A.9 output of show PPES file structure

Listing A.9: PPES File Structure

```
PS ..//> ...\Untitled2.ps1
 [PPes]
    [1]
    20250216152330ppxmlforxsd.xml
    20250216273423ppxmlforxsd.xml
    20250217352315ppxmlforxsd.xml
    20250218152330ppxmlforxsd.xml
    20250218162540ppxmlforxsd.xml
    [10]
    20250224211652ppxmlforxsd.xml
    [2]
    20250216172335ppxmlforxsd.xml
    20250216180134ppxmlforxsd.xml
    20250216191522ppxmlforxsd.xml
    20250216201455ppxmlforxsd.xml
    20250216211418ppxmlforxsd.xml
    [3]
    20250216052303ppxmlforxsd.xml
    20250216132352ppxmlforxsd.xml
    20250216153143ppxmlforxsd.xml
    20250216153515ppxmlforxsd.xml
    20250216171438ppxmlforxsd.xml
    [4]
    20250217180544ppxmlforxsd.xml
    20250217200636ppxmlforxsd.xml
    20250218301539ppxmlforxsd.xml
    20250219273648ppxmlforxsd.xml
    20250221363921ppxmlforxsd.xml
    [5]
    20250221211455ppxmlforxsd.xml
```

```
[6]
20250224214205ppxmlforxsd.xml
[7]
20250224214405ppxmlforxsd.xml
[8]
20250225094842ppxmlforxsd.xml
[9]
20250224211910ppxmlforxsd.xml
[Drone Domain Model]
20250220231035ppxmlforxsd.xml
Drone_Domain Model_2.xml
Drone_Domain_Model.xml
```